# RANCANGAN SOLUSI BIG DATA UNTUK KURASI DAN LISENSI KATALOG WEB NOVEL

**GOLDWIN HOXENLLY**
**2702749771**




**PROGRAM STUDI MAGISTER MTI**
**BINUS GRADUATE PROGRAM**
**UNIVERSITAS BINA NUSANTARA**
**JAKARTA**
**2026**

# DAFTAR ISI


BAB 1 PENDAHULUAN ..... 1
1.1 Latar Belakang Masalah ..... 1
1.2 Market dan Business Drivers ..... 2
1.3 Pemangku Kepentingan yang Terlibat ..... 3
BAB 2 RANCANGAN SOLUSI BIG DATA ..... 5
2.1 Metode Pengumpulan Data ..... 5
2.2 Teknologi Penyimpanan Data ..... 7
2.3 Processing Framework ..... 9
BAB 3 TATA KELOLA IT ..... 12
3.1 Security ..... 12
3.2 Compliance ..... 12
3.3 Data Privacy ..... 13
3.4 Data Quality ..... 13
BAB 4 DATA ..... 17
4.1 Deskripsi Data ..... 17
4.1.1 Sumber dan Volume ..... 17
4.1.2 Elemen Data dan Formatnya ..... 17
4.1.3 Pembersihan dan Organisasi Data ..... 18
4.1.4 Exploratory Data Analysis (EDA) ..... 19
4.2 Model Data ..... 24
4.3 Dataset untuk Diolah Menggunakan *Machine Learning* ..... 30
BAB 5 MACHINE LEARNING UNTUK BIG DATA ..... 32
5.1 *Data Preprocessing* ..... 32
5.2 *Feature Engineering* ..... 33
5.3 Algoritma *Machine Learning* ..... 35
5.4 Proses Pengembangan Model dengan dan tanpa Spark MLlib ..... 36
5.5 Evaluasi Model ..... 38
5.6 Analisis ..... 43
BAB 6 GRAPH ANALYTICS ..... 45
6.1 Pengembangan *Graph* ..... 45
6.2 Analisis dan *Insight* yang Diperoleh ..... 46
REFERENSI ..... 1

# DAFTAR GAMBAR

Gambar 1.1 Jumlah judul per tahun rilis di katalog NovelUpdates, 2010 sampai 2025 ..... 1
Gambar 1.2 Sebaran jumlah suara rating per judul ..... 2
Gambar 2.1 Arsitektur solusi Big Data katalog web novel ..... 5
Gambar 3.1 Jumlah baris setelah setiap langkah pembersihan (tabel quality_report) ..... 14
Gambar 3.2 Halaman Quality & Governance di dashboard: ringkasan 23 pemeriksaan dan rincian pilar Security serta Compliance ..... 16
Gambar 4.1 Persentase nilai kosong per kolom pada tier full setelah normalisasi sentinel ..... 20
Gambar 4.2 Distribusi jumlah bab, rating mentah dibanding weighted rating, jumlah suara, dan panjang sinopsis ..... 21
Gambar 4.3 Lima belas genre terbesar beserta rata-rata weighted rating (WR) judul tepercaya ..... 22
Gambar 4.4 Rata-rata weighted rating menurut kelompok panjang dan kecepatan rilis (populasi tepercaya) ..... 23
Gambar 4.5 Matriks korelasi Spearman antarvariabel numerik pada populasi tepercaya ..... 23
Gambar 4.6 Tren rata-rata weighted rating, median jumlah bab, dan persentase judul tamat per tahun rilis ..... 24
Gambar 4.7 Star schema katalog web novel beserta jumlah baris setiap tabel ..... 25
Gambar 4.8 Dashboard rata-rata weighted rating per genre, dengan genre tertinggi ditandai ..... 27
Gambar 4.9 Skema property graph dan dua proyeksi yang dianalisis ..... 28
Gambar 4.10 Peta genre dan peringkat PageRank ..... 29
Gambar 5.1 Pipeline *machine learning* dari zona clean sampai penyajian ..... 32
Gambar 5.2 Kualitas dan waktu latih Head A dengan dan tanpa Spark MLlib ..... 38
Gambar 5.3 Perbandingan empat model Head A pada set uji ..... 39
Gambar 5.4 F1 per genre pada ensemble SVM + Random Forest (set uji) ..... 40
Gambar 5.5 Kurva objective MLlib per iterasi (kiri) dan log loss latih, validasi, dan uji Head A (kanan) ..... 42
Gambar 5.6 Kurva MSE latih dan uji Head D per jumlah pohon ..... 42
Gambar 6.1 Distribusi derajat simpul pada graf kemiripan (skala log) ..... 46
Gambar 6.2 Empat belas komunitas terbesar graf kemiripan; tebal garis menunjukkan jumlah sisi antarkomunitas (>= 400) ..... 47
Gambar 6.3 Homofili genre di graf kemiripan terhadap F1 per genre; ukuran titik sebanding dengan jumlah judul ..... 48

# DAFTAR TABEL

Tabel 1.1 Market dan business drivers beserta implikasinya pada rancangan solusi ..... 3
Tabel 1.2 Pemangku kepentingan, kepentingan, dan peran solusi ..... 3
Tabel 2.1 Ringkasan metode pengumpulan data ..... 7
Tabel 2.2 Perbandingan opsi penyimpanan dan keputusannya ..... 8
Tabel 4.1 Berkas sumber data dan katalog gabungannya ..... 17
Tabel 4.2 Klasifikasi elemen data berdasarkan format ..... 18
Tabel 4.3 Berkas hasil pemisahan format ..... 19
Tabel 4.4 Statistik deskriptif variabel numerik (tier full) ..... 20
Tabel 4.5 Pemetaan karakteristik data, model data, dan operasi ..... 24
Tabel 4.6 Hasil analisis dua proyeksi graf ..... 28
Tabel 4.7 Elemen data yang dipakai pada tahap *machine learning* ..... 30
Tabel 5.1 Blok fitur pada pipeline *machine learning* ..... 33
Tabel 5.2 Algoritma per tugas dan padanannya yang dapat diskalakan ..... 35
Tabel 5.3 Head A dengan dan tanpa Spark MLlib (protokol validasi yang sama) ..... 37
Tabel 5.4 Hasil Head A pada set uji (protokol Fase 4) ..... 39
Tabel 5.5 Profil tujuh klaster arketipe (Head C) ..... 40
Tabel 5.6 Hasil Head D pada set uji ..... 41
Tabel 5.7 Loss Head A pada set latih, validasi, dan uji ..... 41
Tabel 6.1 Ringkasan graf kemiripan novel ..... 45
Tabel 6.2 Profil delapan komunitas terbesar ..... 47

# DAFTAR PSEUDOCODE

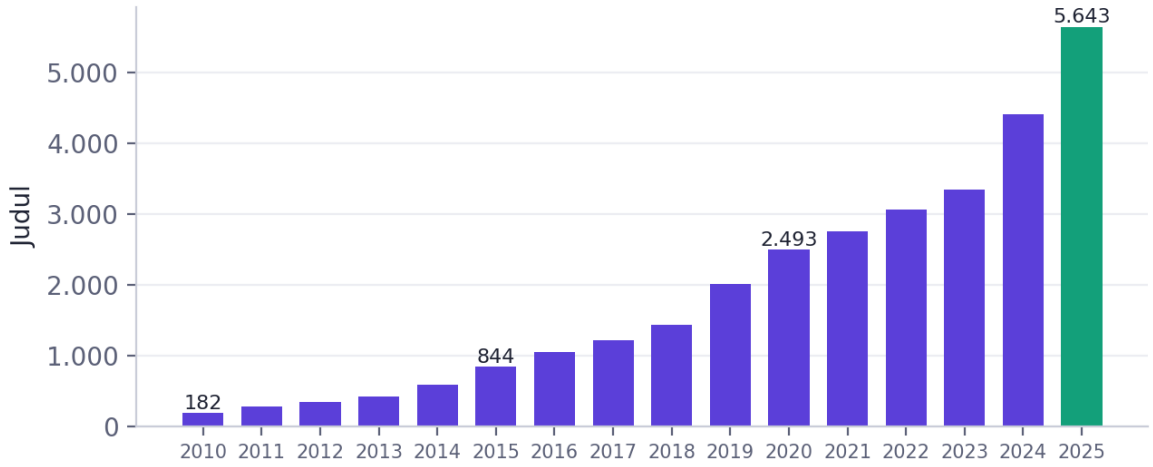
Pseudocode 2.1 Pengambilan halaman dengan jeda adaptif ..... 6
Pseudocode 2.2 Ingestion idempoten dengan manifest checksum ..... 6
Pseudocode 2.3 Ingest inkremental dengan Structured Streaming ..... 7
Pseudocode 2.4 Pemuatan dataset ke HDFS ..... 8
Pseudocode 2.5 Job MapReduce frekuensi dan ko-okurensi genre ..... 9
Pseudocode 2.6 Reduce-side join CSV x JSONL ..... 10
Pseudocode 2.7 Orkestrasi dengan retry dan resume ..... 10
Pseudocode 3.1 Pemeriksaan keamanan object storage ..... 12
Pseudocode 3.2 Pemeriksaan provenance dan versioning ..... 13
Pseudocode 3.3 Pemindaian PII dalam satu agregasi ..... 13
Pseudocode 3.4 Normalisasi sentinel ..... 14
Pseudocode 3.5 Resolusi alias penulis dengan union-find ..... 15
Pseudocode 3.6 Bayesian weighted rating ..... 15
Pseudocode 4.1 Profil elemen data berbasis registri ..... 17
Pseudocode 4.2 Deduplikasi, array multinilai, dan fitur turunan ..... 18
Pseudocode 4.3 Pemisahan fisik per kelas format ..... 19
Pseudocode 4.4 Alur perhitungan EDA (dari Dataset/variety) ..... 19
Pseudocode 4.5 Pembangunan dimensi, bridge, dan fakta ..... 25
Pseudocode 4.6 Agregasi OLAP di halaman Data Warehouse ..... 26
Pseudocode 4.7 Proyeksi dan analisis graf ..... 28
Pseudocode 5.1 Preprocessing teks, label, dan pembagian data ..... 33
Pseudocode 5.2 Pelatihan Head A tanpa Spark MLlib ..... 37
Pseudocode 5.3 Pelatihan Head A dengan Spark MLlib ..... 37
Pseudocode 6.1 Pembangunan dan analisis graf kemiripan ..... 45

# BAB 1
# PENDAHULUAN

## 1.1 Latar Belakang Masalah

Web novel dari Tiongkok, Korea Selatan, dan Jepang kini dibaca jauh di luar negara asalnya. Karya-karya ini lahir di platform serial seperti Qidian, JJWXC, Munpia, atau Kakao Page, lalu diterjemahkan ke bahasa Inggris oleh platform berlisensi maupun kelompok penerjemah komunitas. Sektor yang dibahas dalam laporan ini adalah media dan hiburan digital, tepatnya penerbitan fiksi daring lintas bahasa. Berbeda dengan buku cetak yang terbit sekali jadi, web novel terbit bab demi bab, sering setiap hari, dan bisa berjalan bertahun-tahun. Satu judul dengan seribu bab bukan hal aneh.

Pertumbuhannya terlihat jelas di katalog NovelUpdates, direktori komunitas terbesar untuk web novel terjemahan. Dari data yang dikumpulkan untuk tugas ini, judul dengan tahun rilis 2015 berjumlah 844, sedangkan judul dengan tahun rilis 2025 mencapai 5.643. Artinya, dalam sepuluh tahun, jumlah judul baru per tahun naik hampir tujuh kali lipat (Gambar 1.1). Hingga katalog dikumpulkan pada awal Oktober 2026, sudah tercatat 2.525 judul untuk tahun 2026, padahal tahun itu belum berakhir.



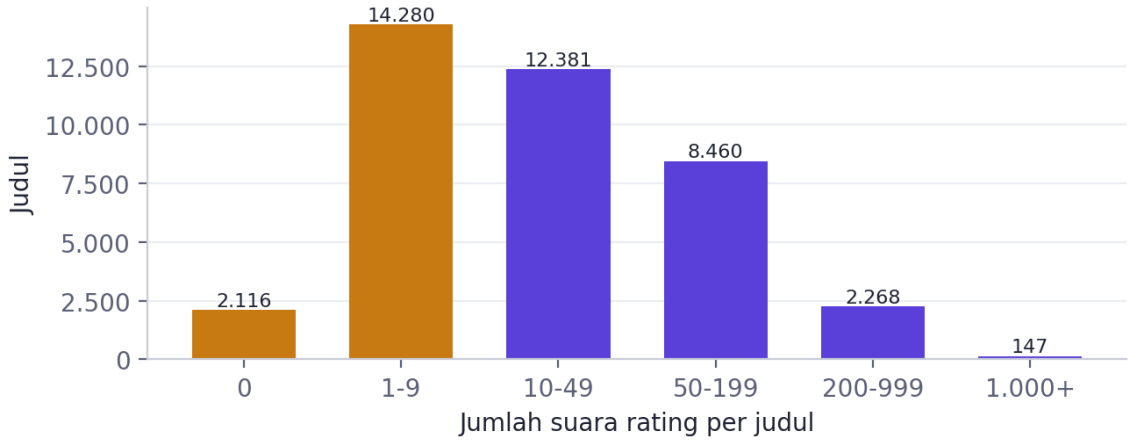

Gambar 1.1 Jumlah judul per tahun rilis di katalog NovelUpdates, 2010 sampai 2025

Banjir judul ini menimbulkan masalah bagi pihak yang harus memilih. Platform penerjemahan perlu memutuskan judul mana yang layak dilisensi dan diterjemahkan, padahal menerjemahkan satu judul berarti membayar ratusan bab kerja penerjemah dan editor. Median panjang judul di katalog adalah 135 bab, dengan rata-rata 292 bab, dan 3.804 judul melewati 598 bab. Pembaca menghadapi masalah yang sama dari sisi lain: dari puluhan ribu judul, mana yang cocok dengan selera mereka, dan mana yang layak diikuti sampai tamat?

Ada tiga hambatan yang membuat keputusan itu sulit diambil hanya dengan membaca halaman katalog satu per satu.

- **Sinyal kualitas tidak merata**: Sebanyak 41,4% judul punya kurang dari 10 suara rating, dan 2.116 judul belum dinilai sama sekali (Gambar 1.2). Rata-rata rating mentah dari dua atau tiga suara tidak bisa dibandingkan dengan rating dari ratusan suara. Sebanyak 1.575 judul tercatat bernilai sempurna 5,0, tetapi hampir semuanya berasal dari kurang dari 10 suara; di antara judul dengan minimal 10 suara, hanya 2 judul yang bernilai 5,0. Inilah masalah *cold start* yang klasik: judul baru justru yang paling perlu dinilai, tetapi paling sedikit datanya.

- **Data tersebar dan setengah terstruktur:** Informasi setiap judul ada di halaman HTML yang berbeda. Genre, tag, dan penulis disimpan sebagai daftar berpemisah koma. Status rilis bercampur dengan jumlah bab dalam satu teks bebas, misalnya "208 Chapters (Ongoing)", dan sinopsis berupa prosa bebas. Nilai kosong juga tidak konsisten: ada yang ditulis 0, "Unknown", atau "N/A".
- **Label genre bersifat subjektif:** Satu judul rata-rata menyandang 5,19 genre, dengan median 11 tag. Genre besar seperti Romance dan Fantasy muncul bersama hampir semua genre lain, sehingga label genre saja tidak cukup untuk membedakan judul.




Gambar 1.2 Sebaran jumlah suara rating per judul

Urgensinya datang dari dua arah. Pertama, volume katalog terus tumbuh lebih cepat daripada kapasitas kurator manusia untuk menilainya. Kedua, keputusan lisensi yang keliru mahal biayanya, sementara judul yang tepat bisa menjadi sumber kekayaan intelektual bernilai tinggi (lihat Subbab 1.2). Tanpa sistem yang mengumpulkan, membersihkan, dan menyatukan data katalog secara otomatis, keputusan tetap bertumpu pada intuisi dan popularitas sesaat.

Karena itu, rumusan masalah tugas ini adalah: bagaimana merancang solusi Big Data yang mampu mengumpulkan katalog web novel terjemahan dari sumber web secara berkelanjutan, membersihkan dan memodelkannya sesuai karakter tiap jenis data, lalu menyajikannya sebagai dasar keputusan lisensi, kurasi, dan rekomendasi?

Dari rumusan itu diturunkan tiga tujuan yang menjadi ukuran keberhasilan rancangan. Pertama, seluruh katalog, bukan sampel, dapat dikumpulkan ulang secara berkala tanpa membebani sumber data dan tanpa menggandakan data. Kedua, setiap angka yang dipakai untuk keputusan dapat ditelusuri ke berkas sumbernya, dan aturan kualitasnya diperiksa otomatis di setiap eksekusi. Ketiga, analis bisa menjawab pertanyaan per segmen (genre, asal negara, waktu) dan pertanyaan relasi (genre mana yang berdekatan, penulis mana yang berkolaborasi) tanpa menulis kode baru.

## 1.2 Market dan Business Drivers

Pendorong pasar (*market drivers*) adalah kondisi di luar organisasi yang membuat masalah ini semakin penting, sedangkan pendorong bisnis (*business drivers*) adalah tujuan internal yang ingin dicapai lewat solusi. Keduanya dirangkum pada Tabel 1.1.

Pendorong pertama adalah ekonomi adaptasi kekayaan intelektual. Web novel semakin sering menjadi bahan baku industri hiburan lain. *Solo Leveling*, web novel Korea, diadaptasi menjadi anime yang tayang pada 2024. *Omniscient Reader's Viewpoint* dirilis sebagai film layar lebar pada 2025. Dari Tiongkok, *Mo Dao Zu Shi* menjadi drama *The Untamed* (2019) dan *Lord of the Mysteries* menjadi serial animasi pada 2025. Setiap kisah sukses seperti ini menaikkan nilai hak atas judul yang

belum dilisensi, sehingga kemampuan mengenali judul potensial lebih awal menjadi keunggulan bersaing.

Pendorong kedua adalah persaingan platform terjemahan. Webnovel (Qidian International), Wuxiaworld, dan platform lain bersaing merebut pembaca yang sama. Pendorong ketiga adalah penurunan biaya terjemahan: penerjemahan berbantuan mesin memungkinkan lebih banyak judul masuk ke pasar bahasa Inggris, yang pada gilirannya mempercepat pertumbuhan katalog pada Gambar 1.1. Pendorong keempat adalah komposisi pasokan yang terkonsentrasi. Dari judul yang negara asalnya diketahui, 60,6% berasal dari Tiongkok, 21,9% dari Korea, dan 17,3% dari Jepang. Ketiganya juga berbeda dalam penerimaan pembaca: pada judul dengan rating tepercaya, rata-rata weighted rating judul Tiongkok 3,732, Korea 3,641, dan Jepang 3,599 (Subbab 4.1.4). Platform yang ingin tumbuh perlu memahami perbedaan selera dan kualitas antar-asal ini.

Dari sisi organisasi, solusi ini ditujukan untuk empat hal: menurunkan risiko keputusan lisensi dengan dasar data yang terukur; menaikkan retensi pembaca lewat rekomendasi yang relevan; mendeteksi tren genre dan asal negara lebih awal; serta mengurangi kerja manual kurator dengan pipeline yang berjalan otomatis dan bisa diulang.

Tabel 1.1 Market dan business drivers beserta implikasinya pada rancangan solusi

| Jenis | Pendorong | Implikasi pada solusi |
|---|---|---|
| Market | Adaptasi IP ke anime, drama, dan film | Perlu metrik popularitas dan kualitas yang bisa dibandingkan lintas judul (weighted rating, reading list) |
| Market | Persaingan platform terjemahan | Perlu rekomendasi dan penemuan judul yang lebih baik dari pesaing |
| Market | Biaya terjemahan turun, katalog membengkak | Arsitektur harus tahan pertumbuhan volume dan kedatangan data bertahap |
| Market | Pasokan terkonsentrasi di CN, KR, JP | Analisis harus bisa dipotong per asal negara, genre, dan waktu |
| Bisnis | Menurunkan risiko lisensi | Star schema untuk analisis OLAP per segmen |
| Bisnis | Menaikkan retensi pembaca | Model rekomendasi berbasis konten dan analisis relasi genre |
| Bisnis | Deteksi tren lebih awal | Ingest inkremental dan dimensi waktu |
| Bisnis | Efisiensi kurasi | Pipeline otomatis dengan audit kualitas di setiap eksekusi |

## 1.3 Pemangku Kepentingan yang Terlibat

Pemangku kepentingan dipetakan berdasarkan kepentingan mereka terhadap data dan bagaimana solusi melayani kepentingan itu (Tabel 1.2). Pengguna utama solusi adalah tim lisensi dan tim editorial di platform penerjemahan. Pembaca adalah penerima manfaat tidak langsung, sedangkan penulis asli, penerbit asal, dan NovelUpdates adalah pemilik sumber data yang hak-haknya harus dihormati (lihat Bab 3).

Tingkat pengaruh dan kepentingan mereka tidak sama. Tim lisensi dan tim data berpengaruh tinggi sekaligus berkepentingan tinggi, sehingga kebutuhan merekalah yang menentukan bentuk star schema dan dashboard. NovelUpdates berpengaruh tinggi walaupun tidak memakai solusi, karena akses ke sumber data bergantung pada kebijakan mereka. Itulah alasan etika scraping dimasukkan sebagai syarat rancangan, bukan tambahan.

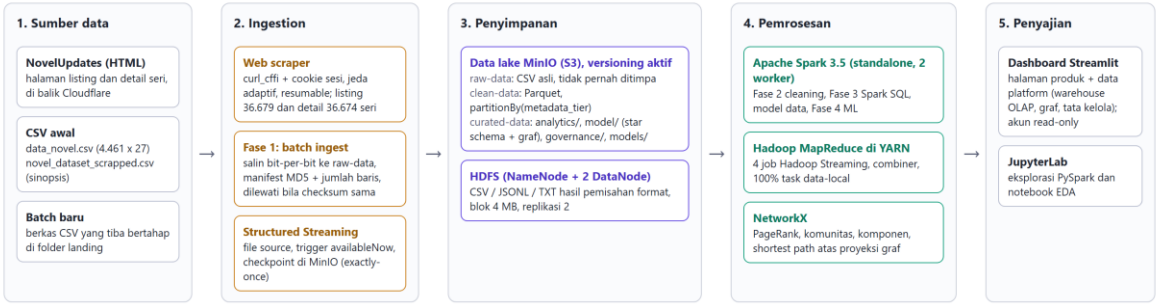
Tabel 1.2 Pemangku kepentingan, kepentingan, dan peran solusi

| Pemangku kepentingan | Kepentingan | Yang disediakan solusi |
|---|---|---|
| Tim lisensi dan akuisisi platform | Memilih judul yang layak dibeli haknya | Analisis segmen genre, asal, dan waktu di dashboard warehouse |
| Editor dan kurator konten | Menyusun daftar rekomendasi dan promosi | Peringkat berbasis weighted rating, peta relasi genre |

| Pemangku kepentingan | Kepentingan | Yang disediakan solusi |
|---|---|---|
| Penerjemah | Mengetahui panjang dan status karya sebelum berkomitmen | Atribut jumlah bab, status tamat, kecepatan rilis |
| Pembaca | Menemukan judul yang sesuai selera | Rekomendasi berbasis konten di halaman AI Studio |
| Penulis asli dan penerbit asal (Qidian, JJWXC, Munpia, dll.) | Karya dan nama mereka dipakai secara wajar | Hanya metadata dan sinopsis untuk analitik nonkomersial; nama pena diperlakukan sebagai kredit karya |
| NovelUpdates (pemilik sumber) | Server tidak terbebani, data tidak disalahgunakan | Scraper dengan jeda 6 sampai 45 detik, tanpa permintaan paralel |
| Tim data (engineer, analis, data scientist) | Pipeline yang andal dan bisa diaudit | Orkestrasi dengan retry dan resume, audit tata kelola otomatis |
| Tim legal dan kepatuhan | Kepatuhan hak cipta dan pelindungan data pribadi | Provenance berbasis checksum, zona raw yang tidak pernah ditimpa, pemindaian PII |

# BAB 2
# RANCANGAN SOLUSI BIG DATA

Rancangan solusi disusun dalam lima lapisan: sumber data, ingestion, penyimpanan, pemrosesan, dan penyajian (Gambar 2.1). Seluruh komponen berjalan sebagai container Docker dalam satu jaringan internal, sehingga lingkungan yang sama bisa dijalankan ulang di mesin lain dengan satu perintah.




Gambar 2.1 Arsitektur solusi Big Data katalog web novel

## 2.1 Metode Pengumpulan Data

Data dikumpulkan lewat empat jalur yang saling melengkapi (Tabel 2.1).

### 1. Dataset awal (batch)

Titik awal proyek adalah dua berkas CSV hasil kompilasi katalog publik NovelUpdates: `data_novel.csv` berisi 4.461 judul dengan 27 kolom metadata, dan `novel_dataset_scrapped.csv` berisi 5.000 baris sinopsis. Keduanya dihubungkan lewat kolom `url`. Sebelum join, 538 baris `url` ganda pada berkas sinopsis harus dibuang; tanpa langkah ini tabel hasil join menggelembung menjadi 4.999 baris dan semua agregat menjadi salah.

### 2. Web scraping bertahap

Katalog 4.461 judul ternyata hanya sebagian kecil dari NovelUpdates, jadi scraper ditulis untuk mengambil sisanya dalam dua tahap. Tahap *listing* mengambil 36.679 judul unik dari halaman daftar seri; 35.209 di antaranya belum ada di dataset awal, sehingga katalog gabungan berisi 39.670 baris sebelum pembersihan akhir. Tahap *detail* membuka halaman setiap seri dan mengambil 27 kolom yang sama dengan dataset awal; hasilnya 36.674 halaman berhasil diambil (99,99%). Lima sisanya selalu mengembalikan HTTP 404 karena seri tersebut sudah dihapus atau berganti judul.

Karena NovelUpdates berada di balik Cloudflare, scraper memakai `curl_cffi` yang meniru sidik jari TLS peramban, ditambah cookie sesi dari peramban pengguna. Ada empat keputusan desain yang penting di sini:

- **Jeda adaptif:** Jeda dasar dimulai dari 6 detik, dengan jeda acak antara nilai dasar dan 1,67 kali nilai dasar. Setiap HTTP 429 mengalikan jeda dasar dengan 1,6 sampai batas 45 detik, dan setiap respons normal menurunkannya perlahan dengan faktor 0,93. Percobaan awal dengan jeda tetap 2,5 sampai 5 detik lancar selama sekitar 40 halaman, lalu dibalas 429 terus-menerus.
- **Cooldown berkala:** Setiap 40 permintaan, scraper beristirahat 30 sampai 60 detik, meniru jeda alami pembaca manusia.
- **Resumable:** Setiap halaman yang berhasil langsung ditulis ke CSV. Menjalankan ulang scraper otomatis melewati URL yang sudah ada, sehingga proses yang secara total memakan sekitar 100 jam (kira-kira 10 detik per halaman) bisa dihentikan dan dilanjutkan kapan saja.

- **Prioritas nilai:** Judul dari papan peringkat populer diambil lebih dulu karena merekalah yang punya cukup suara untuk dianalisis. Jika proses berhenti di tengah jalan, data yang sudah terkumpul tetap berguna.

Logika pengambilan satu halaman dirangkum pada Pseudocode 2.1. Perhatikan bahwa HTTP 403 atau halaman tantangan Cloudflare menghentikan proses, bukan diulang. Mengulang permintaan yang ditolak hanya menambah beban server tanpa peluang berhasil, karena penyebabnya adalah cookie yang kedaluwarsa.

**Pseudocode 2.1 Pengambilan halaman dengan jeda adaptif**

```
// source: scripts/scraper/novelupdates.py
CONST MIN_DELAY = 6, MAX_DELAY = 10, MAX_ADAPTIVE = 45
CONST STEP_UP = 1.6, STEP_DOWN = 0.93, COOLDOWN_EVERY = 40
FUNCTION fetch_page(url, max_attempts = 4):
    FOR attempt = 1 TO max_attempts:
        sleep(random(delay, delay * MAX_DELAY / MIN_DELAY))
        IF request_count MOD COOLDOWN_EVERY = 0:
            sleep(random(30, 60))               // periodic long pause
        response <- GET(url, timeout = (15, 40)) // (connect, read)
        IF response.status = 200:
            IF "Just a moment" IN response.text:
                STOP "cf_clearance cookie expired"
            delay <- MAX(delay * STEP_DOWN, MIN_DELAY)
            RETURN response.text
        IF response.status = 403:
            STOP "cookie or User-Agent mismatch"
        IF response.status = 429:
            delay <- MIN(delay * STEP_UP, MAX_ADAPTIVE)
            sleep(random(45, 90) * attempt)     // rate-limit specific backoff
            CONTINUE
        sleep(5 * attempt)                      // other errors: retry
    RETURN NULL                                 // permanent failure, URL skipped
// run_detail: FOR EACH url (popular first) NOT YET IN detail.csv:
//     append_row(detail.csv, parse_detail(fetch_page(url)))   -> resumable
```

### 3. Ingestion ke data lake

Sejak 7 Oktober 2026 keempat berkas sumber disatukan dulu oleh skrip `consolidate_catalogue.py` menjadi satu katalog, `novel_catalogue.csv` (39.670 baris, 28 kolom). Untuk setiap seri, data halaman detail diutamakan, lalu data halaman daftar, dan dataset awal hanya dipakai untuk sekitar 3 ribu seri yang tidak tercakup scraping. Katalog inilah satu-satunya masukan pipeline, dan Fase 1 menyalinnya secara bit-per-bit ke zona `raw-data`. Untuk setiap berkas dibuat *manifest* JSON berisi checksum MD5, jumlah baris, dan waktu unggah. Jika checksum berkas lokal sama dengan yang sudah tersimpan, unggahan dilewati. Jika berbeda, pipeline mencatat perubahan checksum lama ke baru di log sebelum menimpa, sehingga versi sebelumnya tetap bisa dilacak lewat versioning bucket. Dengan cara ini ingestion bersifat *idempotent*, yaitu menjalankannya berkali-kali tidak menggandakan data (Pseudocode 2.2).

**Pseudocode 2.2 Ingestion idempoten dengan manifest checksum**

```
// source: src/phases/phase1_collect.py
FUNCTION ingest_file(source, object_key, force = FALSE):
    md5 <- compute_md5(source, chunk = 1 MB)
    rows <- count_csv_rows(source)
    old_manifest <- read_manifest(raw-data, object_key + "_manifest.json")
    IF old_manifest EXISTS AND old_manifest.md5 = md5 AND NOT force:
        log("identical object already in lake, upload skipped")
        RETURN old_manifest
    IF old_manifest EXISTS AND old_manifest.md5 != md5:
        log("source changed: old md5 -> new md5")
    upload(source, raw-data, object_key)
    manifest <- {source, md5, rows, object_uri, uploaded_at_utc}
    write_json(raw-data, object_key + "_manifest.json", manifest)
    RETURN manifest
```

### 4. Ingest inkremental (streaming)

Hasil scraping pada kenyataannya datang bertahap, bukan sekaligus. Untuk kebutuhan ini dipakai Spark Structured Streaming dengan *file source* yang memantau folder `streaming/landing/` di

MinIO, *trigger* `availableNow`, dan checkpoint yang juga disimpan di MinIO. Setiap micro-batch melewati validasi ringkas: baris tanpa `series_id` dibuang, penanda rating tepercaya dikonversi ke boolean, dan waktu ingest dicatat (Pseudocode 2.3).

Perilakunya diuji langsung, tidak diasumsikan dari dokumentasi: batch pertama berisi 500 baris menghasilkan 500 baris baru; batch kedua berisi 500 baris menambah total menjadi 1.000; menjalankan ulang job tanpa berkas baru menghasilkan 0 baris baru. Checkpoint mencegah berkas yang sama diproses dua kali, yang merupakan definisi ingest *exactly-once*. Trigger `availableNow` dipilih karena job berhenti sendiri setelah semua berkas baru diproses, sehingga bisa dijadwalkan seperti job batch tetapi dengan semantik streaming.

**Pseudocode 2.3 Ingest inkremental dengan Structured Streaming**

```
// source: src/phases/streaming_ingest.py
stream <- spark.readStream
            .format("csv").schema(METRICS_SCHEMA)
            .option("maxFilesPerTrigger", 1)         // one file = one micro-batch
            .load("s3a://raw-data/streaming/landing/")
FUNCTION validate(batch):
    batch <- FILTER batch WHERE series_id IS NOT NULL
    batch.is_rated_reliable <- (lower(is_rated_reliable) = "true")
    batch.ingested_at_utc <- now()
    RETURN batch
query <- validate(stream).writeStream
            .format("parquet")
            .option("checkpointLocation", "s3a://curated-data/streaming/_checkpoint")
            .trigger(availableNow = TRUE)             // process all new files, then stop
            .start("s3a://curated-data/streaming/novel_metrics_incremental")
query.awaitTermination()
// files whose offsets are already in the checkpoint are never read again
```

Tabel 2.1 Ringkasan metode pengumpulan data

| Jalur | Sumber | Mekanisme | Volume | Frekuensi |
|---|---|---|---|---|
| Dataset awal | CSV katalog dan sinopsis | Unggah batch + manifest MD5 | 4.461 + 5.000 baris | Sekali, idempoten |
| Scraping listing | Halaman daftar NovelUpdates | `curl_cffi`, jeda adaptif | 36.679 judul unik | Berkala |
| Scraping detail | Halaman seri NovelUpdates | `curl_cffi`, resumable | 36.674 halaman | Berkala, bertahap |
| Streaming | Berkas CSV di folder landing | Structured Streaming, checkpoint | Per batch (uji: 2 x 500 baris) | Setiap batch baru tiba |

## 2.2 Teknologi Penyimpanan Data

Penyimpanan dirancang dua tingkat dengan peran berbeda: data lake berbasis object storage sebagai sumber kebenaran, dan HDFS sebagai penyimpanan terdistribusi yang berdampingan dengan komputasi MapReduce. Model data warehouse (star schema) dan graf tidak dijalankan di DBMS terpisah, melainkan dimaterialisasi sebagai tabel Parquet di zona kurasi. Opsi penyimpanan yang dipertimbangkan beserta keputusannya dirangkum pada Tabel 2.2.

### a. Data lake MinIO

MinIO adalah object storage yang kompatibel dengan API Amazon S3, sehingga Spark membacanya lewat konektor `s3a://` yang sama seperti di layanan cloud. Jika sistem ini dipindah ke AWS S3 atau Google Cloud Storage, kode pipeline tidak perlu diubah; cukup endpoint dan kredensial. Data lake dibagi menjadi tiga zona (ukuran per 15 Oktober 2026):

- `raw-data` (61,7 MB, 4 objek): katalog sumber apa adanya beserta manifest-nya, ditambah folder `streaming/landing/` untuk ingest inkremental. Zona ini tidak pernah ditimpa oleh transformasi apa pun, sehingga setiap hasil bisa ditelusuri kembali ke berkas aslinya.

- `clean-data` (116,3 MB, 3 objek): data hasil pembersihan dalam format Parquet terkompresi Snappy, dipartisi fisik berdasarkan `metadata_tier` (direktori `full` dan `listing`).
- `curated-data` (457,2 MB, 116 objek): hasil analitik Fase 3, star schema dan tabel graf di `model/`, hasil audit di `governance/`, artefak model di `models/`, serta checkpoint Spark dari Fase 5.

Versioning diaktifkan di ketiga bucket agar objek yang tertimpa masih bisa dipulihkan dan diaudit. Aturan lifecycle menghapus versi lama setelah 30 hari di `raw-data` dan 7 hari di dua bucket lainnya, tetapi satu versi sebelumnya selalu disimpan, sehingga lake tidak terus membengkak oleh setiap eksekusi ulang pipeline.

Parquet dipilih untuk zona clean dan curated karena format kolumnar cocok dengan pola baca analitik: kueri yang hanya butuh `weighted_rating` dan `type_origin` tidak perlu membaca kolom sinopsis yang besar. Partisi fisik menambah keuntungan kedua. Kueri yang memfilter `metadata_tier = 'full'` hanya membuka direktori `metadata_tier=full`, dan hal ini terbukti dari munculnya `PartitionFilters` pada physical plan Spark, bukan sekadar `DataFilters` yang diterapkan setelah seluruh berkas dibaca. Untuk itu data ditulis dengan `repartition("metadata_tier")` lalu `partitionBy("metadata_tier")`, sehingga setiap nilai partisi menjadi satu direktori bergaya Hive.

**b. HDFS**

Klaster HDFS terdiri dari satu NameNode dan dua worker. Setiap worker menjalankan DataNode dan NodeManager YARN dalam container yang sama. Ke HDFS dimuat tiga berkas hasil pemisahan format (Subbab 4.1.3): CSV, JSONL, dan TXT yang dipecah ke empat shard. Ukuran blok diatur 4 MB supaya berkas berukuran puluhan megabita tetap terpecah menjadi beberapa blok (enam berkas fisik menjadi 21 blok), dengan faktor replikasi 2 sehingga satu DataNode boleh mati tanpa kehilangan data. Hasil `hdfs fsck` menunjukkan 0 blok kekurangan replika. Skrip pemuatannya menunggu NameNode keluar dari *safe mode* dan jumlah DataNode hidup memenuhi faktor replikasi sebelum menulis (Pseudocode 2.4).

Menempatkan DataNode dan NodeManager pada hostname yang sama bukan detail kecil. Pada konfigurasi awal keduanya terpisah, dan counter Hadoop mencatat 0% task *data-local*. Setelah digabung, 22 dari 22 task map pada empat job membaca blok dari disk lokalnya sendiri.

**Pseudocode 2.4 Pemuatan dataset ke HDFS**

```
// source: scripts/hdfs/load_hdfs.sh
REPLICATION <- 2 ; BLOCK_SIZE <- 4 MB
REPEAT up to 60 times, 5 seconds apart:
    live <- number of live DataNodes (hdfs dfsadmin -report)
    IF live >= REPLICATION AND safemode = OFF: BREAK
IF live < REPLICATION: FAIL "not enough DataNodes"
make_dirs(/novel/structured, /novel/semistructured, /novel/unstructured/synopsis)
FOR EACH file IN {novel_metrics.csv, novel_labels.jsonl, synopsis-000[0-3].txt}:
    hdfs dfs -D dfs.blocksize=BLOCK_SIZE -D dfs.replication=REPLICATION -put -f file
report <- hdfs fsck /novel -files -blocks -locations
ASSERT report.under_replicated = 0
```

Tabel 2.2 Perbandingan opsi penyimpanan dan keputusannya

| Opsi | Kelebihan untuk kasus ini | Kekurangan | Keputusan |
|---|---|---|---|
| Data lake (MinIO, S3) | Murah, skema fleksibel, zona raw tetap utuh, versioning | Tanpa indeks, perlu engine kueri | Dipakai sebagai sumber kebenaran |
| HDFS | Blok berreplika, data locality untuk MapReduce | Mengelola NameNode, satu titik kegagalan tanpa HA | Dipakai untuk jalur MapReduce |
| Data warehouse (star schema) | Kueri OLAP cepat dan mudah dipahami analis | Butuh pemodelan di depan | Dipakai, dimaterialisasi sebagai Parquet |
| NoSQL dokumen | Cocok untuk data bersarang | Data sudah bisa disimpan sebagai array di Parquet | Tidak dipakai |

| Opsi | Kelebihan untuk kasus ini | Kekurangan | Keputusan |
|---|---|---|---|
| Graph database | Traversal dalam yang efisien | Menambah service; kedalaman kueri hanya 2 hop | Tidak dipakai, tabel simpul/sisi siap dimigrasikan |

## 2.3 Processing Framework

Tiga kerangka pemrosesan dipakai sesuai jenis bebannya.

a. **Apache Spark 3.5 (standalone)**

Spark (Zaharia et al., 2016) adalah mesin utama. Secara bawaan klaster terdiri dari satu master dan satu worker dengan 2 core dan memori 3 GB, dan jumlah worker bisa ditambah dengan `docker compose up --scale spark-worker=N`. Spark menjalankan Fase 2 (pembersihan dengan DataFrame API), Fase 3 (analitik dengan Spark SQL), pembangunan star schema dan proyeksi graf, Structured Streaming, serta persiapan data untuk machine learning. Spark dipilih karena satu engine melayani batch, SQL, streaming, dan ML dengan API yang sama. Pemrosesan di memori juga cocok untuk pipeline yang menjalankan puluhan transformasi berurutan. Driver dan executor memakai image Docker yang sama, sehingga versi Python dan PySpark di kedua sisi dijamin identik.

b. **Hadoop MapReduce di YARN**

Sebagai jalur kedua yang independen, empat job Hadoop Streaming dengan mapper dan reducer Python murni dijalankan di atas data HDFS, mengikuti model pemrograman MapReduce (Dean & Ghemawat, 2008). Setiap job mewakili satu kelas format:

- `wordcount` menghitung frekuensi kata pada korpus sinopsis (TXT, tidak terstruktur).
- `genre_stats` menghitung frekuensi dan ko-okurensi genre (JSONL, semi-terstruktur).
- `origin_stats` menghitung statistik per negara asal (CSV, terstruktur).
- `genre_rating` menghitung rata-rata rating per genre dengan *reduce-side join* antara CSV dan JSONL.

Pseudocode 2.5 memperlihatkan job `genre_stats`. Mapper mengeluarkan dua jenis kunci berprefiks dalam satu job, sehingga berkas JSONL cukup dibaca sekali. Reducer yang sama dipakai sebagai combiner, karena penjumlahan bersifat asosiatif dan komutatif: menjumlah hasil parsial di sisi map lalu menjumlahkannya lagi di reducer memberi hasil yang sama persis. Rata-rata tidak bersifat demikian, sehingga job yang butuh rata-rata mengeluarkan pasangan (jumlah, cacah) dan baru membagi di akhir.

**Pseudocode 2.5 Job MapReduce frekuensi dan ko-okurensi genre**

```
// source: src/mapreduce/genre_mapper.py, sum_reducer.py
MAPPER(jsonl_line):
    genres <- sort(unique(json(jsonl_line).genres))
    FOR EACH g IN genres:
        EMIT("G:" + g, 1)
    FOR EACH (a, b) IN combinations(genres, 2):       // a < b, each pair once
        EMIT("P:" + a + "|" + b, 1)
COMBINER = REDUCER(key, values):                      // input already sorted by key
    EMIT(key, SUM(values))
```

Job `genre_rating` memerlukan data dari dua berkas: rating ada di CSV, genre ada di JSONL. Mapper mengenali asal setiap baris dari path berkas input, lalu menandai rekamannya dengan `R` atau `G`. Fase shuffle mengirim semua rekaman ber-`series_id` sama ke reducer yang sama, dan di sanalah keduanya digabung (Pseudocode 2.6). Pilihan *reduce-side join* dipakai karena kedua sisi berukuran sebanding; *map-side join* hanya tepat bila salah satu sisi cukup kecil untuk dimuat ke memori setiap mapper.

**Pseudocode 2.6 Reduce-side join CSV x JSONL**

```
// source: src/mapreduce/join_mapper.py, join_reducer.py
MAPPER(line):
    IF input_path CONTAINS "/structured/":            // structured CSV
        IF is_rated_reliable = 1 AND weighted_rating EXISTS:
            EMIT(series_id, ("R", weighted_rating))
    IF input_path CONTAINS "/semistructured/":        // semi-structured JSONL
        EMIT(series_id, ("G", join(sort(genres), "|")))
REDUCER_STAGE_1(series_id, records):
    rating <- value of the record tagged "R"
    genres <- value of the record tagged "G"
    IF rating EXISTS AND genres EXISTS:
        FOR EACH g IN split(genres, "|"): EMIT(g, (rating, 1))
STAGE_2: summing REDUCER per genre -> (rating_sum, count)
         average <- rating_sum / count              // divided at the end, not in the reducer
```

Combiner memangkas lalu lintas shuffle secara drastis, misalnya pada `wordcount` dari 3.415.443 rekaman menjadi 236.329 (turun 93,1%) dan pada `genre_stats` dari 717.552 menjadi 4.207 (turun 99,4%). Hasil keempat job dicocokkan baris demi baris dengan jawaban Spark SQL atas pertanyaan yang sama, dan keempatnya identik tanpa selisih.

c. **NetworkX untuk algoritma graf**

Proyeksi graf dari 858.901 sisi mentah dikerjakan Spark secara terdistribusi. Hasil proyeksinya (35 simpul genre dan 2.574 simpul penulis) cukup kecil untuk dianalisis di memori satu proses dengan NetworkX: PageRank, sentralitas, komponen terhubung, deteksi komunitas, dan jalur terpendek. Algoritma rinci dibahas di Subbab 4.2.

d. **Orkestrasi**

Seluruh langkah dijalankan oleh `run_pipeline.py` secara berurutan: Fase 1 sampai 4, pemisahan format, lalu lapisan tata kelola (katalog elemen, model data, analisis graf, audit). Pipeline berhenti pada kegagalan pertama, karena melanjutkan di atas data yang tidak lengkap akan menghasilkan analitik yang tampak sukses tetapi salah. Opsi `--retry N` mengulang langkah yang gagal karena gangguan sementara, misalnya NameNode yang masih dalam *safe mode*. Opsi `--resume` melanjutkan dari langkah yang gagal terakhir berdasarkan berkas status yang hanya berlaku untuk argumen eksekusi yang sama (Pseudocode 2.7). Pada pengukuran 6 Oktober 2026, eksekusi Fase 1 sampai 3, pemisahan format, dan seluruh lapisan tata kelola dalam satu sesi Spark bersama selesai dalam 126,9 detik.

**Pseudocode 2.7 Orkestrasi dengan retry dan resume**

```
// source: scripts/run_pipeline.py
FUNCTION run_step(name, func, retries):
    FOR attempt = 1 TO retries + 1:
        TRY: func(); RETURN TRUE
        CATCH error:
            IF attempt <= retries: log(error); sleep(5); CONTINUE
            log_error(error); RETURN FALSE
FUNCTION run_pipeline(start, end, retries, resume):
    signature <- hash(start, end, governance_options)
    done <- IF resume: load_state(signature) ELSE {}
    FOR EACH (name, func) IN [phases start..end, variety, catalog, model, graph, audit]:
        IF name IN done: SKIP
        IF NOT run_step(name, func, retries):
            STOP pipeline              // never continue on incomplete data
        done <- done + {name}; save_state(signature, done)
    clear_state()                      // all steps succeeded, nothing to resume
```

e. **Penyajian**

Hasil akhir dibaca oleh dashboard Streamlit dengan akun MinIO khusus baca. Selain halaman produk (ikhtisar katalog, analitik, AI Studio, metodologi), dashboard memiliki enam halaman *data platform*: tabel katalog lengkap, status data lake, penjelajah OLAP di atas star schema, peta relasi graf, graf kemiripan konten (Bab 6), dan ringkasan tata kelola (Subbab 3.4 dan 4.2). Dua halaman pertama disembunyikan pada mode publik. Dashboard tidak

menjalankan job Spark sendiri; ia hanya membaca tabel Parquet yang sudah dimaterialisasi, sehingga interaksi terasa instan dan angka di layar selalu sama dengan angka di log pipeline.

# BAB 3
# TATA KELOLA IT

## 3.1 Security

Kebijakan keamanan mengikuti prinsip hak akses minimum dan pertahanan berlapis.

- **Bucket tertutup untuk akses anonim:** Ketiga bucket MinIO tidak memiliki policy anonim (SEC-03, lulus untuk ketiganya).
- **Pemisahan hak baca dan tulis:** Dashboard memakai akun `novel-dashboard` yang hanya boleh membaca. Kontrol ini sudah diuji: percobaan menulis dari container dashboard ditolak dengan `AccessDenied`. Bug di lapisan presentasi tidak bisa merusak data kurasi (SEC-04).
- **Isolasi jaringan:** Semua service berada dalam bridge network Docker `bigdata-net`. Hanya port antarmuka pengguna yang dipetakan ke host, dan hanya pada alamat 127.0.0.1 (SEC-05).

Dua temuan yang pada Assignment I berstatus peringatan kini sudah ditutup. SEC-01: kredensial root MinIO bawaan (`minioadmin`) sudah diganti dengan kredensial khusus, dan audit memeriksa bahwa setiap rahasia minimal 16 karakter dan bukan nilai contoh. SEC-02: lalu lintas ke MinIO kini dienkripsi TLS 1.3. Audit tidak hanya membaca skema URL, tetapi melakukan handshake sungguhan yang memverifikasi sertifikat server terhadap CA proyek; Spark memakai CA yang sama lewat truststore Java. Pseudocode 3.1 memperlihatkan bagaimana pemeriksaan ini dijalankan terhadap konfigurasi yang sebenarnya, bukan dari daftar klaim.

**Pseudocode 3.1 Pemeriksaan keamanan object storage**

```
// source: src/phases/governance_audit.py
FUNCTION audit_security():
    add("SEC-01", WARN IF a secret is a default, "change-me..." or < 16 chars ELSE PASS)
    add("SEC-02", PASS IF a TLS handshake verifies the server certificate ELSE WARN)
    FOR EACH bucket IN {raw-data, clean-data, curated-data}:
        policy <- get_bucket_policy(bucket)          // no policy = closed
        public <- ANY statement WITH Effect = "Allow" AND Principal = "*"
        add("SEC-03/" + bucket, FAIL IF public ELSE PASS)
    add("SEC-04", PASS IF MINIO_READONLY_USER account is defined ELSE WARN)
    add("SEC-05", INFO, "only UI ports are mapped, bound to 127.0.0.1")
```

## 3.2 Compliance

Kepatuhan pada proyek ini menyangkut tiga hal: hak cipta, etika pengambilan data, dan jejak audit.

**a. Hak cipta**

Metadata katalog seperti judul, genre, jumlah bab, dan rating bersifat faktual. Sinopsis tetap merupakan ciptaan pemiliknya yang dilindungi Undang-Undang Nomor 28 Tahun 2014 tentang Hak Cipta. Sinopsis hanya dipakai untuk analitik dan pelatihan model nonkomersial dalam konteks akademik, dan dataset tidak dipublikasikan ulang. Status lisensi redistribusi harus ditegaskan terlebih dahulu jika dataset ini akan dibagikan (CMP-04).

**b. Etika scraping**

Scraper tidak pernah mengirim permintaan paralel. Jedanya minimal 6 detik, naik otomatis ketika server memberi sinyal beban (HTTP 429), dan diselingi cooldown berkala (Pseudocode 2.1). Scraper juga berhenti sendiri ketika sesi kedaluwarsa, bukan mencoba menembusnya berulang kali. Dengan rata-rata sekitar 10 detik per halaman, beban yang diberikan setara dengan satu pembaca yang membuka halaman satu per satu.

**c. Jejak audit dan provenance**

Setiap berkas di zona raw memiliki manifest berisi checksum MD5, jumlah baris, dan waktu unggah (CMP-01). Zona raw tidak pernah ditimpa transformasi; semua hasil ditulis ke bucket lain (CMP-02). Versioning aktif di ketiga bucket sehingga versi objek sebelumnya dapat dipulihkan (CMP-03); versi yang lebih lama dihapus otomatis oleh aturan lifecycle (Subbab 2.2). Pemeriksaan provenance dijalankan dengan membaca kembali manifest yang tersimpan di lake (Pseudocode 3.2).

**Pseudocode 3.2 Pemeriksaan provenance dan versioning**

```
// source: src/phases/governance_audit.py
FUNCTION audit_compliance():
    manifests <- ALL objects in raw-data ending with "_manifest.json"
    complete <- EVERY m IN manifests HAS md5, rows, uploaded_at
    add("CMP-01", PASS IF manifests NOT EMPTY AND complete ELSE FAIL,
        evidence = list(name, md5[0..8], rows))
    add("CMP-02", PASS, "transformations only write to clean-data / curated-data")
    status <- {b: get_bucket_versioning(b) FOR b IN buckets}
    add("CMP-03", PASS IF ALL status = "Enabled" ELSE WARN)
    add("CMP-04", INFO, "public source; synopses remain their owners' copyright")
```

## 3.3 Data Privacy

Dataset ini tidak memuat data tingkat individu pembaca. Sinyal pembaca hanya tersedia sebagai agregat (`rating_votes`, `reading_list_count`), sehingga tidak ada subjek data yang bisa diidentifikasi ulang (PRV-03). Pemindaian pola email, nomor telepon Indonesia, nomor kartu kredit, NIK 16 digit, dan alamat IP pada seluruh kolom teks tidak menemukan satu pun kecocokan langsung (PRV-01). Semua kolom dan pola dipindai dalam satu agregasi Spark, bukan satu job per kolom, sehingga pemindaian 39.670 baris katalog sumber selesai dalam hitungan detik (Pseudocode 3.3).

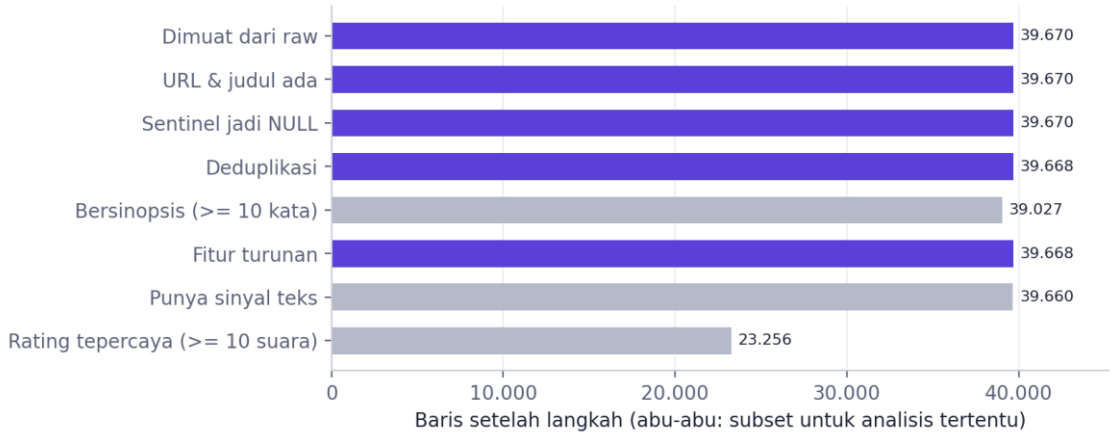
**Pseudocode 3.3 Pemindaian PII dalam satu agregasi**

```
// source: src/phases/governance_audit.py
PATTERNS <- {
    email        : "[\w.+-]+@[\w-]+\.[A-Za-z]{2,}",
    phone        : "(?<!\d)(?:\+62|08)\d{8,12}(?!\d)",
    credit_card  : "(?<!\d)(?:\d[ -]?){13,16}(?!\d)",
    nik_16_digit : "(?<!\d)\d{16}(?!\d)",
    ip_address   : "(?<!\d)(?:\d{1,3}\.){3}\d{1,3}(?!\d)" }
FUNCTION scan_pii(df):
    text_cols <- columns of df with type string
    aggregations <- [ SUM(1 IF df[c] MATCHES PATTERNS[p] ELSE 0) AS c||p
                      FOR c IN text_cols, p IN PATTERNS ]
    result <- df.agg(aggregations)                    // a single Spark job
    RETURN [(c, p, n) FOR (c||p, n) IN result IF n > 0]
```

Satu hal tetap perlu diperhatikan: kolom `authors` memuat nama pena. Nama pena dipublikasikan platform sebagai kredit karya, tetapi tetap dapat menjadi data pribadi jika bisa dikaitkan ke orang nyata, sebagaimana dimaksud Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi. Karena itu nama pena hanya dipakai sebagai atribut karya (siapa menulis apa), tidak dipadankan dengan sumber lain untuk mengungkap identitas asli, dan tidak dipakai untuk profil perorangan. Prinsip minimisasi data juga diterapkan: kolom `artists` (82,36% kosong) dan `english_publisher` (93,28% kosong) dibuang di Fase 2 karena tidak memberi nilai analitik.

## 3.4 Data Quality

Kualitas data dijaga pada lima dimensi: kelengkapan, validitas, keunikan, konsistensi, dan keterpercayaan. Setiap langkah pembersihan mencatat jumlah baris sesudahnya ke tabel `quality_report` (Gambar 3.1), sehingga perubahan jumlah baris yang tidak terduga langsung terlihat.




Gambar 3.1 Jumlah baris setelah setiap langkah pembersihan (tabel quality_report)

### a. Sentinel menjadi NULL

Sumber data memakai angka 0 dan teks "Unknown" sebagai pengganti nilai kosong. Pada dataset awal, 20,22% `chapter_count` dan 22,66% `year` bernilai 0. Tanpa normalisasi, rata-rata jumlah bab ikut menghitung 902 judul bernilai 0 dan meleset sekitar 20%. Rating 0 tanpa suara berarti "belum dinilai", bukan "nilainya jelek", sehingga juga diubah menjadi NULL agar tidak menarik turun rata-rata genre. Satu kasus menarik ditangani di sini: jumlah bab yang terpotong pemisah ribuan ("1,593 Chapters" tercatat sebagai 593) dipulihkan dari kolom `status_coo`, tetapi hanya bila tiga digit terakhirnya sama. Syarat ini mencegah perbedaan wajar antara status lama dan kolom terbaru ikut tertimpa (Pseudocode 3.4).

**Pseudocode 3.4 Normalisasi sentinel**

```
// source: src/phases/phase2_clean.py, normalize_sentinels
FOR EACH col IN {type, type_origin, tags, language, authors, original_publisher,
                 status_coo, release_frequency} THAT EXIST IN df:
    df[col] <- NULL IF lower(trim(df[col])) IN {"unknown", "n/a"} ELSE trim(df[col])
status_chapters <- extract_number(remove_commas(status_coo), "(\d+)\s*chapters?")
df.chapter_count <- status_chapters IF status_chapters > chapter_count > 0
                                   AND status_chapters MOD 1000 = chapter_count
                    ELSE chapter_count               // restore counts truncated at the comma
df.chapter_count <- NULL IF chapter_count <= 0
df.year <- NULL IF year < 1000 OR year > 2030          // 1740 (a classic) stays valid
df.reading_list_count <- NULL IF reading_list_count < 0
df.rating_score <- NULL IF rating_score <= 0 OR rating_votes IS NULL OR rating_votes <= 0
```

### b. Temuan baru dari star schema

Saat membangun dashboard warehouse, tiga cacat kualitas yang sebelumnya lolos audit ditemukan dan diperbaiki pada 29 September 2026:

- Teks "N/A" dari halaman detail belum dikenali sebagai sentinel. Akibatnya 478 judul punya asal negara "N/A", 1.140 judul punya penulis "N/A" (sehingga "N/A" menjadi penulis paling produktif di `dim_author`), dan 4.916 judul punya penerbit "N/A".
- Tujuh judul punya tahun rilis terpotong seperti 15, 21, 202, dan 206. Batas bawah tahun dinaikkan menjadi 1000, bukan 1900, karena *Painted Skin* (1740, Pu Songling) adalah karya klasik yang sah.
- Empat judul punya `reading_list_count` negatif (terendah -162), yang membuat `log1p` pada skor popularitas menghasilkan NaN.

Pemeriksaan QLT-03 juga diperketat. Versi lama hanya mengecek angka 0, itulah sebabnya "N/A" lolos dengan status lulus. Versi baru ikut memeriksa sentinel teks, tahun di bawah 1000, dan nilai negatif. Pelajaran dari temuan ini adalah bahwa aturan kualitas perlu diuji dari sisi

konsumen data juga: cacat "N/A" tidak terlihat di log pembersihan, tetapi langsung terlihat begitu tabel dimensi penulis diurutkan.

**c. Keunikan dan konsistensi**

Deduplikasi memakai `series_id`, bukan judul, karena ada judul kembar. Bila satu seri muncul lebih dari sekali, baris dengan suara terbanyak dipertahankan karena itu snapshot paling matang. Hasil audit: 39.668 kunci unik dari 39.668 baris (QLT-02).

Nama penulis yang sama dalam dua aksara, seperti "Mizume, ミズメ", disatukan lewat resolusi entitas. Dua nama dianggap alias bila salah satunya beraksara CJK, Hangul, atau Kana dan yang lain tidak, serta nama yang lebih jarang tidak pernah muncul tanpa nama yang lebih sering. Hasilnya, 36.391 nama menjadi 27.324 entitas. Sebanyak 364 komponen yang memuat dua atau lebih nama beraksara asli sengaja tidak digabung, karena melebur dua orang berbeda lebih merugikan daripada melewatkan satu penggabungan (Pseudocode 3.5).

**Pseudocode 3.5 Resolusi alias penulis dengan union-find**

```
// source: src/phases/data_model.py
count[name] <- number of titles per author name
FOR EACH pair (a, b) appearing on the same title, with shared count n:
    different_script <- (a is CJK/Hangul/Kana) XOR (b is CJK/Hangul/Kana)
    contained        <- n = MIN(count[a], count[b])  // the rarer name never appears alone
    IF different_script AND contained: union(a, b)
FOR EACH component K produced by union-find:
    native <- names in K written in a native script
    IF |K| = 1:          resolution <- "single"
    IF |native| = 1:     resolution <- "merged"; canonical <- most frequent romanisation in K
    ELSE:                resolution <- "ambiguous"   // >= 2 native names: deliberately not
merged
```

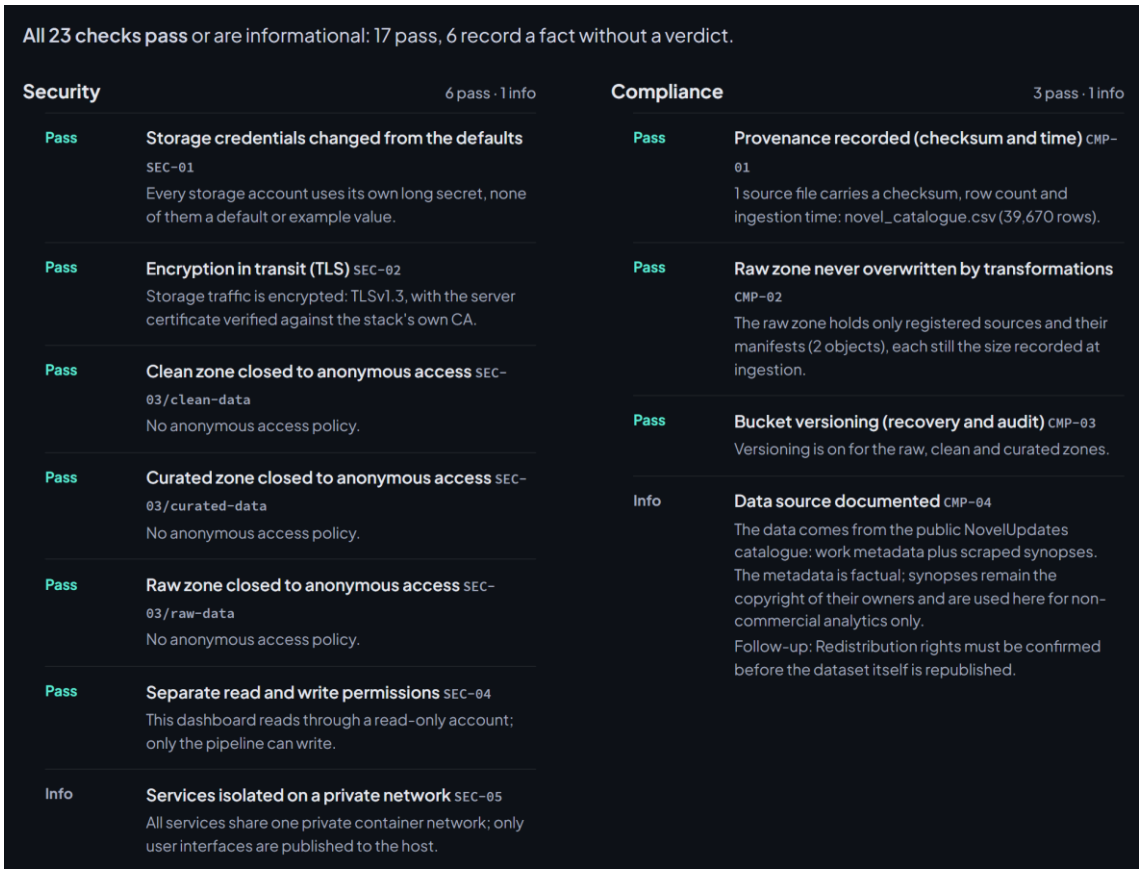
**d. Keterpercayaan rating**

Rating ditampilkan sebagai *weighted rating* Bayesian dengan rumus WR = (v / (v + m)) x R + (m / (v + m)) x C, dengan R rating mentah, v jumlah suara, m ambang suara (10), dan C rata-rata global pada populasi tepercaya (3,6519). Judul bernilai 5,0 dari 1 suara ditarik mendekati rata-rata, sedangkan judul bernilai 4,4 dari 280 suara nyaris tidak bergeser. Hanya 23.256 judul (58,6%) yang punya minimal 10 suara, dan populasi inilah yang dipakai untuk peringkat (Pseudocode 3.6). Cakupan sinopsis pada tier metadata lengkap mencapai 98,38% (QLT-04). Hasil seluruh pemeriksaan tata kelola dapat dilihat di halaman Quality & Governance pada dashboard (Gambar 3.2): dari 23 pemeriksaan, 17 lulus, 6 bersifat informatif, dan tidak ada yang gagal.

**Pseudocode 3.6 Bayesian weighted rating**

```
// source: src/phases/phase2_clean.py, add_weighted_rating
m <- 10
reliable <- FILTER df WHERE rating_score IS NOT NULL AND rating_votes >= m
C <- MEAN(reliable.rating_score)                      // 3.6519 over 23,256 titles
IF C IS NULL: FAIL "no reliable ratings"
v <- df.rating_votes
```

```
df.weighted_rating <- ROUND((v / (v + m)) * rating_score + (m / (v + m)) * C, 4)
                      IF rating_score IS NOT NULL ELSE NULL
df.is_rated_reliable <- rating_score IS NOT NULL AND v >= m
```




Gambar 3.2 Halaman Quality & Governance di dashboard: ringkasan 23 pemeriksaan dan rincian pilar Security serta Compliance

# BAB 4
# DATA

## 4.1 Deskripsi Data

### *4.1.1 Sumber dan Volume*

Tabel 4.1 merangkum empat berkas sumber yang dikumpulkan dan katalog gabungannya, yang sejak 7 Oktober 2026 menjadi satu-satunya berkas sumber di zona raw. Setelah digabung, dideduplikasi, dan dibersihkan, katalog berisi 39.668 judul: 39.652 bermetadata lengkap (tier `full`) dan 16 hanya bermodal teks dan genre (tier `listing`). Per 15 Oktober 2026, total data di tiga zona lake adalah 635,3 MB dalam 123 objek; lebih dari separuhnya adalah artefak model dan checkpoint di zona curated. Semua angka katalog dalam laporan ini, dari Bab 1 sampai Bab 6, berasal dari satu snapshot yang sama, yaitu zona clean hasil eksekusi pipeline 7 Oktober 2026. Pengecualiannya hanya uraian kejadian yang diberi tanggal sendiri, seperti temuan 29 September 2026 di Subbab 3.4.

Volume ini perlu dinyatakan dengan jujur. Sebanyak 635 MB masih muat di satu mesin, sehingga dari sisi ukuran data proyek ini belum termasuk "big". Yang dirancang untuk skala besar adalah arsitekturnya: penyimpanan terdistribusi, pemrosesan terdistribusi, ingest inkremental, dan partisi fisik. Jika katalog tumbuh sepuluh kali lipat, yang perlu ditambah adalah jumlah worker, bukan kode pipeline.

Tabel 4.1 Berkas sumber data dan katalog gabungannya

| Berkas | Isi | Ukuran | Baris |
|---|---|---|---|
| `data_novel.csv` | Katalog awal, 27 kolom metadata | 2,44 MB | 4.461 |
| `novel_dataset_scrapped.csv` | Sinopsis, dihubungkan lewat `url` | 4,89 MB | 5.000 |
| `scraped_listing.csv` | Hasil scraping halaman daftar seri | 42,57 MB | 36.679 |
| `scraped_detail.csv` | Hasil scraping halaman detail seri | 24,72 MB | 36.674 |
| `novel_catalogue.csv` | Katalog gabungan keempat berkas; satu-satunya berkas sumber di zona raw sejak 7 Oktober 2026 | 61,62 MB | 39.670 |

### *4.1.2 Elemen Data dan Formatnya*

Modul `data_catalog.py` menginventarisasi 28 elemen data dan mengklasifikasikan formatnya. Klasifikasi ditentukan dari struktur isi, bukan dari tipe fisik. Contohnya, `genres` dan `type` sama-sama bertipe string, tetapi `genres` berisi daftar multinilai sehingga termasuk semi-terstruktur. Karena itu kelas format ditulis eksplisit di sebuah registri beserta alasannya, lalu statistiknya (nilai unik, persentase sentinel, contoh nilai) diukur dari data (Pseudocode 4.1). Kolom sumber yang belum terdaftar di registri memicu peringatan, sehingga kolom baru dari scraper tidak bisa lolos tanpa diklasifikasikan. Hasilnya: 20 elemen terstruktur, 6 semi-terstruktur, dan 2 tidak terstruktur (Tabel 4.2).

**Pseudocode 4.1 Profil elemen data berbasis registri**

```
// source: src/phases/data_catalog.py
REGISTRY <- [(name, source, format_class, domain, sentinel_rule, rationale), ...]  // 28
elements
FUNCTION build_catalog(raw):
    result <- []
    FOR EACH e IN REGISTRY:
        column <- raw[e.name]
        n_sentinel <- COUNT rows WHERE sentinel_condition(e.rule)(column)
        result.append({e.name, e.format_class, e.domain,
                       n_distinct = COUNT_DISTINCT(column),
                       pct_sentinel = 100 * n_sentinel / row_count,
                       sample = first_non_sentinel_value(column), e.rationale})
```

```
    unregistered <- columns(raw) - names(REGISTRY)
    IF unregistered NOT EMPTY: warn("not yet classified", unregistered)
    RETURN result
```

Tabel 4.2 Klasifikasi elemen data berdasarkan format

| Kelas format | Jumlah | Elemen | Ciri |
|---|---|---|---|
| Terstruktur | 20 | `series_id`, `type`, `type_origin`, `language`, `year`, `chapter_count`, `rating_score`, `rating_votes`, `reading_list_count`, lima kolom peringkat, `is_completed`, `licensed`, `completely_translated`, `original_publisher`, `artists`, `english_publisher` | Nilai atomik, satu nilai per sel |
| Semi-terstruktur | 6 | `genres`, `tags`, `authors`, `url`, `status_coo`, `release_frequency` | Ada struktur di dalam string: daftar berkoma, path URL, atau pola angka dan satuan |
| Tidak terstruktur | 2 | `synopsis`, `title` | Prosa bebas; median sinopsis 133 kata |

Beberapa elemen semi-terstruktur menyimpan informasi yang baru bisa dipakai setelah diurai. `status_coo` ("1,593 Chapters (Completed)") menjadi cadangan untuk memulihkan `chapter_count` yang terpotong pemisah ribuan. Kasus seperti ini ditemukan satu kali pada dataset awal; pada katalog gabungan, halaman detail sudah menyimpan angka lengkapnya sehingga aturan ini tidak mengubah baris apa pun. `release_frequency` ("Every 0.9 Day(s)") diurai menjadi `days_per_chapter`, yang lalu dikelompokkan menjadi lima kategori kecepatan rilis. Kolom `authors` juga lebih rumit dari kelihatannya: 14.685 judul mencantumkan dua nama penulis atau lebih, tetapi setelah resolusi alias hanya 1.802 judul yang benar-benar ditulis bersama. Sisanya adalah satu orang yang ditulis dalam dua aksara.

### *4.1.3 Pembersihan dan Organisasi Data*

Urutan langkah pembersihan terlihat pada Gambar 3.1. Katalog `novel_catalogue.csv` dibaca dari zona raw dengan skema eksplisit, baris tanpa URL atau judul dibuang, dan setiap baris diberi `metadata_tier`: `full` untuk seri yang halaman detailnya berhasil diambil dan `listing` untuk 16 seri yang hanya punya teks dan genre. Seri `listing` belum punya `series_id`, sehingga diberi id sintetis negatif yang deterministik agar tidak terbuang saat deduplikasi dan tidak mungkin bertabrakan dengan id asli. Berikutnya sentinel diubah menjadi NULL, baris dideduplikasi berdasarkan `series_id`, dan kolom `artists` serta `english_publisher` yang lebih dari 80% kosong dibuang. Setelah itu kolom multinilai diurai, teks diproses, weighted rating dihitung, dan fitur turunan dibentuk. Pada langkah sentinel, peringkat aktivitas mingguan dan bulanan yang bernilai tepat #2000 juga diubah menjadi NULL, karena NovelUpdates memotong peringkat di angka itu: nilai #2000 berarti di luar 2000 besar, bukan peringkat sungguhan. Mencampurnya dengan peringkat asli tidak memicu error apa pun, tetapi membuat perbandingan peringkat menjadi bias.

Kolom multinilai dipecah menjadi array yang bersih: dipisah per koma, dipangkas spasinya, dibuang elemen kosongnya, dan diunikkan. Genre pertama dijadikan `primary_genre` untuk visualisasi, dan kombinasi genre diurutkan agar {Action, Romance} dan {Romance, Action} dihitung sebagai kombinasi yang sama. Teks sinopsis ditokenisasi dengan pipeline Spark ML (RegexTokenizer lalu StopWordsRemover dengan 191 stopword bahasa Inggris dan istilah domain). Pseudocode 4.2 merangkum deduplikasi dan fitur turunan.

**Pseudocode 4.2 Deduplikasi, array multinilai, dan fitur turunan**

```
// source: src/phases/phase2_clean.py
window <- PARTITION BY series_id ORDER BY rating_votes DESC, reading_list_count DESC
df <- FILTER df WHERE row_number() OVER window = 1          // most mature snapshot
```

```
FUNCTION to_array(text): RETURN unique(drop_empty(trim(split(text, ","))))
df.genre_list <- to_array(genres); df.tag_list <- to_array(tags)
df.primary_genre <- genre_list[0]
df.genre_combo <- join(sort(genre_list), " + ")
df.days_per_chapter <- extract_number(release_frequency, "([\d.]+)\s*day")
df.release_pace <- BUCKET(days_per_chapter; <=2 Daily, <=7 Weekly, <=30 Monthly,
                          <=180 Slow, otherwise Stalled)
df.length_bucket <- BUCKET(chapter_count; <50, <200, <500, <1000, 1000+)
df.decade <- FLOOR(year / 10) * 10
df.popularity_score <- log1p(rating_votes) + log1p(reading_list_count)
                       IF metadata_tier = "full" ELSE NULL
```

Zona clean menyimpan ketiga kelas format dalam satu tabel Parquet. Agar variasi format juga nyata di tingkat berkas, `data_variety.py` memecahnya menjadi tiga berkas dengan format yang sesuai sifatnya (Tabel 4.3). Kolom atomik ditulis ke CSV; daftar genre, tag, penulis, serta status dan kecepatan rilis ditulis sebagai objek JSON bersarang; sinopsis ditulis sebagai teks biasa yang dipecah ke empat shard untuk menghindari masalah berkas kecil di HDFS. Setiap berkas dicatat dalam manifest dengan MD5, lalu dibaca ulang untuk memastikan jumlah rekamannya sama dengan sumber (Pseudocode 4.3). Ketiga berkas inilah yang dimuat ke HDFS dan diolah MapReduce.

**Pseudocode 4.3 Pemisahan fisik per kelas format**

```
// source: src/phases/data_variety.py
clean <- spark.read.parquet("s3a://clean-data/novels_clean")
write_csv("structured/novel_metrics.csv", clean[ATOMIC_COLUMNS])          // 23 columns
FOR EACH row IN clean:
    write_jsonl("semistructured/novel_labels.jsonl",
        {series_id, title, url, genres: [..], tags: [..], authors: [..],
         status: {raw, chapter_count, is_completed},
         release: {raw, days_per_chapter}})
with_synopsis <- FILTER clean WHERE synopsis IS NOT EMPTY
FOR EACH shard i IN 0..3:
    write_txt("unstructured/synopsis/synopsis-000i.txt",
              lines "series_id<TAB>title<TAB>synopsis" for shard i)
manifest <- {file, class, format, size, md5, record_count}
ASSERT reread(csv) = 39,668 AND reread(jsonl) = 39,668 AND reread(txt) = 39,342
```

Tabel 4.3 Berkas hasil pemisahan format

| Kelas | Berkas | Format | Baris |
|---|---|---|---|
| Terstruktur | `structured/novel_metrics.csv` | CSV, 23 kolom | 39.668 |
| Semi-terstruktur | `semistructured/novel_labels.jsonl` | JSONL, genre/tag/penulis sebagai array | 39.668 |
| Tidak terstruktur | `unstructured/synopsis/synopsis-*.txt` | TXT, 4 shard | 39.342 (yang bersinopsis) |

Ringkasan isi katalog setelah pembersihan: 55,5% judul sudah tamat, 15,1% dari judul yang status lisensinya tercatat berlisensi resmi (kolom ini kosong pada lebih dari separuh katalog), dan 20,4% selesai diterjemahkan. Terdapat 35 genre dan 772 tag. Genre terbanyak adalah Romance (25.080 judul) dan Fantasy (23.122 judul).

### *4.1.4 Exploratory Data Analysis (EDA)*

EDA dilakukan untuk memahami bentuk data sebelum memilih model data dan metrik. Semua angka di subbab ini dihitung dari ekspor zona clean (`Dataset/variety/`, hasil eksekusi 7 Oktober 2026). Kecuali disebutkan lain, analisis rating memakai populasi tepercaya (23.256 judul dengan minimal 10 suara), sedangkan analisis kelengkapan dan distribusi memakai tier `full` (39.652 judul). Langkah perhitungannya dirangkum pada Pseudocode 4.4.

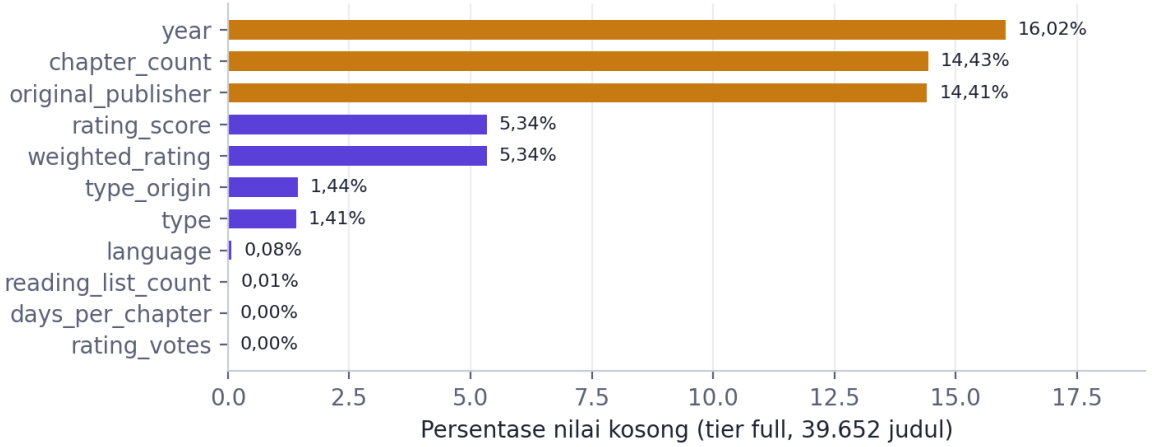
**Pseudocode 4.4 Alur perhitungan EDA (dari Dataset/variety)**

```
m <- read_csv("structured/novel_metrics.csv"); full <- FILTER m WHERE metadata_tier = "full"
labels <- read_jsonl("semistructured/novel_labels.jsonl")
reliable <- FILTER full WHERE is_rated_reliable = 1
missing[k] <- 100 * MEAN(full[k] IS NULL) FOR k IN key_columns
stats[k] <- {count, mean, std, min, Q1, median, Q3, max, skewness} FOR k IN numeric_columns
chapter_upper <- Q3 + 1.5 * (Q3 - Q1); outliers <- COUNT(chapter_count > chapter_upper)
```

```
genres <- explode(labels.genres) JOIN reliable ON series_id
rating_by_genre <- GROUP genres BY genre: MEAN(weighted_rating), COUNT   // at least 300 titles
rating_by_length <- GROUP reliable BY length_bucket(chapter_count): MEAN(weighted_rating)
rating_by_pace <- GROUP reliable BY pace_bucket(days_per_chapter): MEAN(weighted_rating)
correlation <- SPEARMAN(reliable[weighted_rating, log1p(votes), log1p(reading_list),
                                 log1p(chapters), log1p(days_per_chapter), year])
words <- tokens [a-z]{3,} across all synopsis shards, stopwords removed; vocabulary <-
COUNT_DISTINCT(words)
```

### a. Kelengkapan data

Gambar 4.1 menunjukkan persentase nilai kosong per kolom setelah sentinel dinormalisasi. Kolom jumlah suara, reading list, dan kecepatan rilis hampir selalu terisi. Tiga kolom perlu perhatian: `year` kosong pada 16,02% judul, `chapter_count` pada 14,43%, dan `original_publisher` pada 14,41%. Implikasinya langsung pada analisis: setiap tren per tahun hanya mewakili sekitar 84,0% katalog, dan setiap rata-rata jumlah bab hanya mewakili 85,6%. Menganggap nilai kosong itu sebagai 0, seperti yang dilakukan sumber data, akan menggeser hasil secara sistematis. Rating kosong pada 5,34% judul adalah judul yang belum pernah dinilai.




Gambar 4.1 Persentase nilai kosong per kolom pada tier full setelah normalisasi sentinel

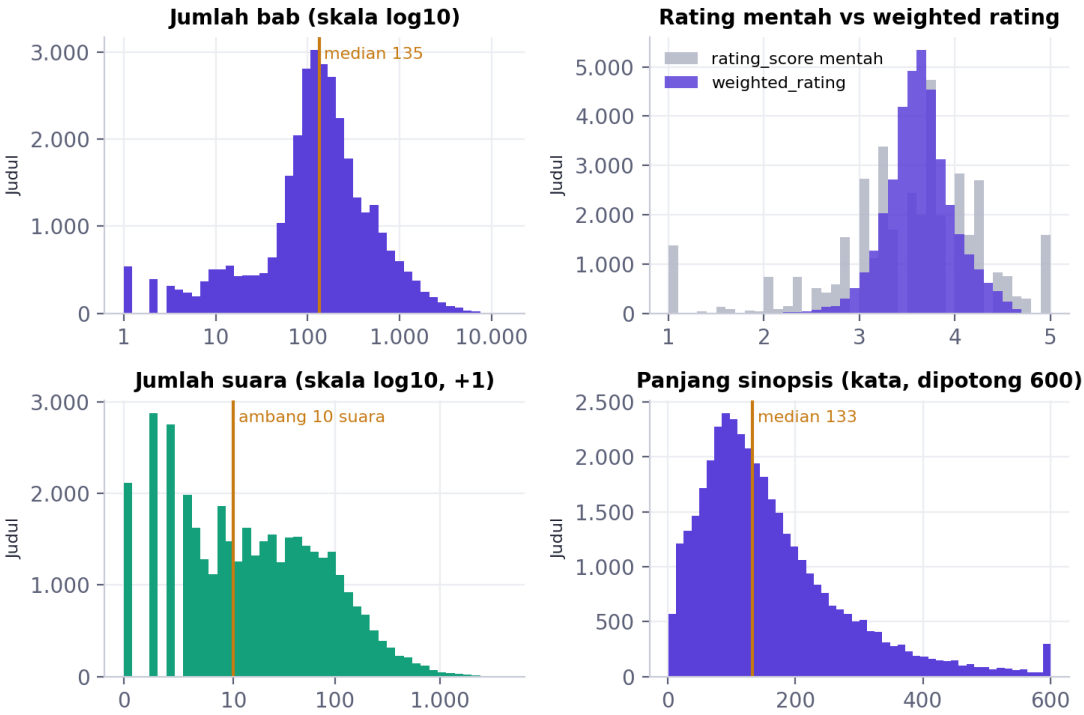
### b. Distribusi variabel numerik

Statistik deskriptif pada Tabel 4.4 memperlihatkan bahwa hampir semua variabel numerik sangat menceng ke kanan. Jumlah bab punya median 135 tetapi rata-rata 291,7 dan maksimum 14.276, dengan skewness 7,52. Dengan aturan IQR dari Tukey (1977), batas atasnya Q3 + 1,5 x IQR = 598 bab, dan sebanyak 3.804 judul (11,21% dari yang jumlah babnya diketahui) tergolong outlier panjang. Judul-judul ini tidak dibuang karena mereka nyata, tetapi karena itulah median dan skala logaritma dipakai di analisis berikutnya, bukan rata-rata dan skala linear. Jumlah suara dan reading list menceng lebih ekstrem lagi (skewness 9,45 dan 4,88), yang menjelaskan mengapa korelasi dihitung dengan Spearman atas nilai log.

Tabel 4.4 Statistik deskriptif variabel numerik (tier full)

| Variabel | n | Rata-rata | Std | Min | Median | Q3 | Maks | Skewness |
|---|---|---|---|---|---|---|---|---|
| chapter_count | 33.931 | 291,69 | 563,39 | 1 | 135 | 280 | 14.276 | 7,52 |
| rating_score | 37.536 | 3,49 | 0,82 | 1,0 | 3,6 | 4,0 | 5,0 | -0,95 |
| weighted_rating | 37.536 | 3,64 | 0,35 | 1,81 | 3,62 | 3,85 | 4,88 | -0,01 |
| rating_votes | 39.652 | 56,97 | 141,44 | 0 | 15 | 56 | 4.225 | 9,45 |
| reading_list_count | 39.648 | 975,84 | 2.046,44 | 1 | 200 | 948 | 44.569 | 4,88 |
| days_per_chapter | 39.651 | 218,94 | 769,19 | 0,1 | 25,7 | 101,3 | 20.610 | 8,23 |
| year | 33.300 | 2021,09 | 5,14 | 1740 | 2022 | 2024 | 2026 | -8,29 |

Gambar 4.2 memperlihatkan empat distribusi kunci. Panel kanan atas paling penting untuk rancangan metrik. Rating mentah punya lonjakan tajam di 1,0 dan 5,0 yang berasal dari judul bersuara sedikit: 1.575 judul bernilai 5,0 dari kurang dari 10 suara, sementara hanya 2 judul yang bernilai 5,0 dengan minimal 10 suara. Setelah weighted rating diterapkan, lonjakan itu hilang, simpangan baku turun dari 0,821 menjadi 0,351, dan distribusinya hampir simetris (skewness -0,01). Karena itu rating mentah tidak dipakai untuk peringkat. Panel kiri bawah menunjukkan bahwa median suara hanya 15, dengan garis ambang 10 suara membelah katalog menjadi dua bagian yang hampir sama besar.




Gambar 4.2 Distribusi jumlah bab, rating mentah dibanding weighted rating, jumlah suara, dan panjang sinopsis

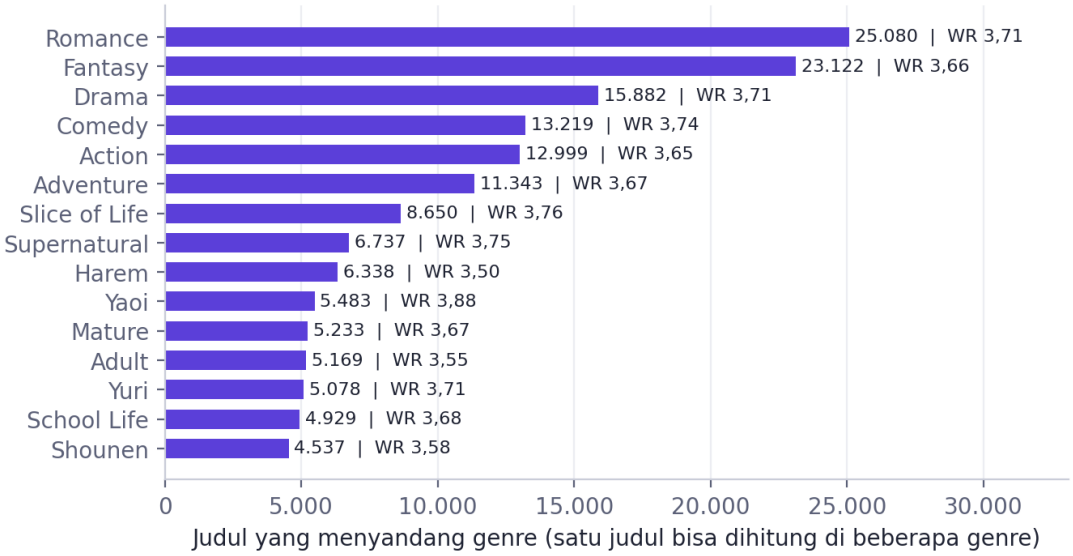
### c. Data tidak terstruktur: sinopsis

Sebanyak 39.342 judul punya sinopsis, dengan median 133 kata, rata-rata 161,3 kata, dan persentil ke-90 sebesar 314 kata. Sekitar 11,9% sinopsis kurang dari 50 kata, terlalu pendek untuk dijadikan satu-satunya dasar klasifikasi. Setelah tokenisasi (huruf kecil, minimal tiga huruf) dan pembuangan 191 stopword, kosakata korpus berisi 62.412 kata unik. Angka ini sama persis dengan jumlah baris keluaran job MapReduce `wordcount`, yang menjadi pemeriksaan silang independen antara jalur Python lokal dan jalur Hadoop. Kata paling sering adalah "one", "world", "life", "even", dan "like". Kata-kata ini generik dan muncul di hampir semua genre, sehingga pada tahap pemodelan bobot TF-IDF dipakai untuk menurunkan pengaruh kata yang terlalu umum.

### d. Data semi-terstruktur: genre dan tag

Setiap judul menyandang rata-rata 5,19 genre dan 15,43 tag (median 11) dari 772 tag berbeda. Tag paling sering bersifat deskriptif tokoh dan latar: Male Protagonist (14.090), Female Protagonist (13.774), Transmigration (10.671), dan Handsome Male Lead (10.143). Gambar 4.3 menampilkan 15 genre terbesar beserta rata-rata weighted rating-nya. Genre terbanyak ternyata bukan genre dengan rating tertinggi. Romance menyandang 25.080 judul (63,2% katalog) dengan rata-rata 3,71. Di antara genre dengan minimal 300 judul tepercaya, rata-rata tertinggi dimiliki Shounen Ai (3,904), Yaoi (3,879), Historical (3,814), Mystery (3,808), dan Horror

(3,784), sedangkan terendah adalah Ecchi (3,438), Harem (3,503), Adult (3,554), Xuanhuan (3,564), dan Shounen (3,583). Karena satu judul dihitung di beberapa genre, jumlah semua batang melebihi total katalog; fakta inilah yang mendorong penggunaan bridge table di model data (Subbab 4.2).



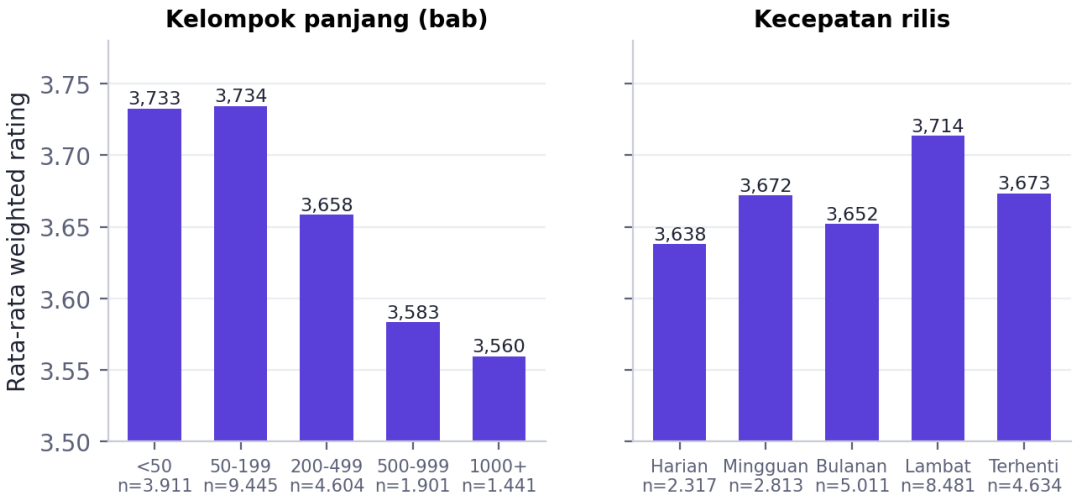

Gambar 4.3 Lima belas genre terbesar beserta rata-rata weighted rating (WR) judul tepercaya

**e. Variabel kategorikal lain**

Pada populasi tepercaya, judul yang sudah tamat rata-rata dinilai lebih tinggi (3,738) daripada yang masih berjalan (3,593), dan judul berlisensi resmi sedikit lebih tinggi (3,734) daripada yang tidak (3,672). Menurut asal negara, judul Tiongkok unggul (3,732, n = 12.456) atas Korea (3,641, n = 5.544) dan Jepang (3,599, n = 5.133). Format didominasi Web Novel (36.832 judul), disusul Light Novel (1.574) dan Published Novel (686).

**f. Panjang dan kecepatan rilis terhadap rating**

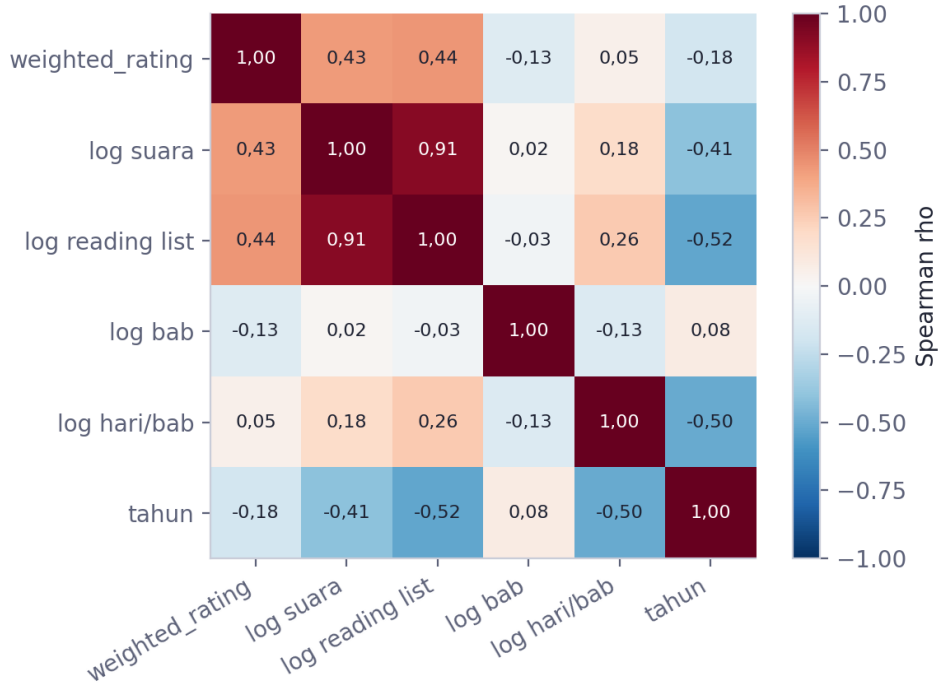
Gambar 4.4 menguji dua dugaan yang sering muncul di komunitas pembaca. Dugaan pertama, judul yang lebih panjang lebih disukai, tidak didukung data. Rata-rata weighted rating stabil sekitar 3,73 untuk judul di bawah 200 bab, lalu turun konsisten ke 3,658 (200 sampai 499 bab), 3,583 (500 sampai 999 bab), dan 3,560 (1.000 bab atau lebih). Dugaan kedua, judul yang rilis lebih cepat lebih disukai, juga tidak didukung: kelompok rilis lambat (1 sampai 6 bulan per bab) justru memiliki rata-rata tertinggi (3,714), dan selisih antarkelompok tidak membentuk pola yang berurutan. Bagi tim lisensi, ini berarti panjang karya dan kecepatan rilis tidak bisa dipakai sebagai pengganti ukuran kualitas.




Gambar 4.4 Rata-rata weighted rating menurut kelompok panjang dan kecepatan rilis (populasi tepercaya)

## g. Korelasi antarvariabel

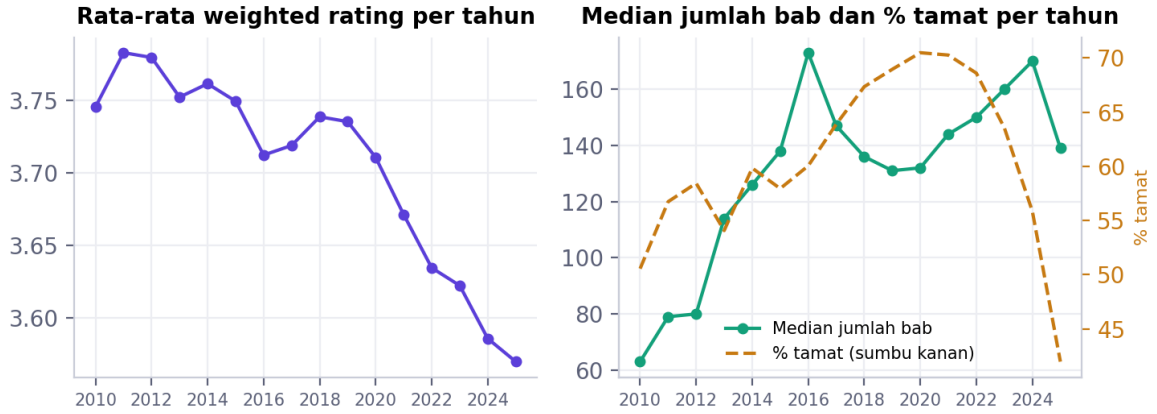
Matriks korelasi Spearman pada Gambar 4.5 dihitung atas nilai log untuk variabel yang menceng. Weighted rating berkorelasi sedang dengan jumlah suara (0,43) dan reading list (0,44): judul yang disukai cenderung juga banyak dibaca. Korelasi rating dengan panjang karya lemah dan negatif (-0,13), sejalan dengan Gambar 4.4. Dua temuan lain penting untuk tahap pemodelan. Pertama, jumlah suara dan reading list berkorelasi sangat kuat (0,91), sehingga keduanya hampir redundan dan tidak boleh diperlakukan sebagai dua sinyal independen. Kedua, tahun rilis berkorelasi negatif dengan reading list (-0,52) dan jumlah suara (-0,41). Judul baru belum sempat mengumpulkan pembaca, sehingga membandingkan popularitas judul lintas tahun tanpa koreksi waktu paparan akan bias terhadap judul lama.




Gambar 4.5 Matriks korelasi Spearman antarvariabel numerik pada populasi tepercaya

### h. Tren waktu

Gambar 4.6 memperlihatkan dua tren sejak 2010. Rata-rata weighted rating per tahun rilis, dihitung atas semua judul yang sudah punya weighted rating, turun dari 3,783 (2011) menjadi 3,570 (2025). Penurunan ini perlu dibaca hati-hati. Sebagian mencerminkan efek paparan yang sama: judul baru bersuara sedikit, sehingga rating mereka lebih banyak ditarik ke arah rata-rata global dan lebih sensitif terhadap beberapa suara awal. Panel kanan menunjukkan median jumlah bab per tahun berkisar 131 sampai 173 bab sejak 2016, sedangkan proporsi judul tamat memuncak pada 70,5% untuk judul tahun 2020 dan turun ke 42,0% untuk judul tahun 2025, karena banyak judul baru memang masih berjalan. Dimensi waktu di star schema dirancang untuk menjawab pertanyaan seperti ini per genre dan per asal negara.




Gambar 4.6 Tren rata-rata weighted rating, median jumlah bab, dan persentase judul tamat per tahun rilis

### i. Kesimpulan EDA untuk rancangan

Temuan di atas langsung memengaruhi keputusan model data. Distribusi yang menceng menuntut median dan skala log di tampilan analitik. Rating mentah yang bias menuntut weighted rating dan filter populasi tepercaya sebagai bawaan. Genre yang multinilai menuntut bridge table agar ukuran tidak tergandakan. Nilai kosong pada tahun dan jumlah bab menuntut dimensi waktu yang hanya berisi tahun sah. Korelasi kuat antara suara dan reading list, serta bias waktu paparan, dicatat sebagai batasan untuk tahap pemodelan di Bab 5.

## 4.2 Model Data

Satu model data tidak cukup untuk katalog ini, karena setiap kelas format menuntut operasi yang berbeda. Tabel 4.5 memetakan karakteristik data ke model dan operasi yang dipilih.

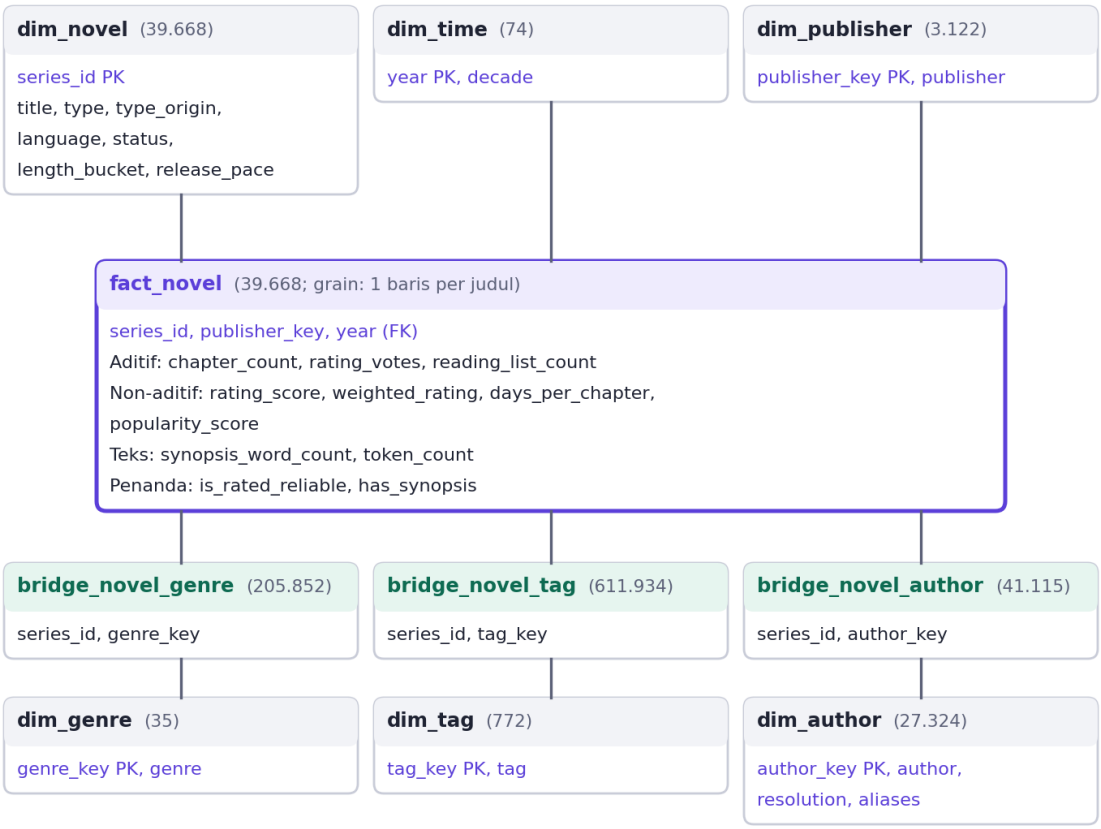
Tabel 4.5 Pemetaan karakteristik data, model data, dan operasi

| Karakteristik data | Model data | Operasi utama |
|---|---|---|
| Metrik dan dimensi atomik (terstruktur) | Star schema: `fact_novel` + 6 tabel dimensi | Filter, star join, GROUP BY, slice, dice, roll-up, drill-down, pivot |
| Genre, tag, penulis multinilai (semi-terstruktur) | Bridge table hasil `explode`; dokumen JSONL | GROUP BY per nilai tanpa `LIKE`, map-reduce per elemen array |
| Relasi antarentitas (novel, genre, tag, penulis) | Property graph: `graph_nodes` + `graph_edges` | Traversal, PageRank, komunitas, komponen terhubung, jalur terpendek |
| Sinopsis prosa (tidak terstruktur) | Matriks fitur TF-IDF dan vektor SVD | Kemiripan kosinus, klasifikasi multilabel, klasterisasi |

### a. Star schema

Tabel fakta `fact_novel` memiliki *grain* satu baris per judul (Gambar 4.7), mengikuti pemodelan dimensional Kimball dan Ross (2013). Grain ini dipilih dengan sengaja. Jika grain dibuat per pasangan judul dan genre, ukuran seperti `rating_votes` akan tergandakan untuk judul bergenre banyak, dan total suara katalog membengkak dua sampai tiga kali lipat. Dimensi bernilai tunggal (`dim_novel`, `dim_time`, `dim_publisher`) terhubung langsung ke fakta. Dimensi bernilai banyak (`dim_genre`, `dim_tag`, `dim_author`) terhubung lewat bridge table. `dim_time` hanya berisi tahun yang sah, sesuai temuan EDA bahwa 16,02% tahun kosong.

Kunci pengganti (*surrogate key*) diberikan secara deterministik, diurutkan menurut jumlah judul lalu nama, bukan dengan `monotonically_increasing_id()`. Dengan cara ini kunci yang sama keluar di setiap eksekusi, sehingga artefak dari dua eksekusi berbeda tetap bisa dibandingkan. Efek sampingnya berguna: `publisher_key <= 15` berarti tepat 15 penerbit terbesar (Pseudocode 4.5).




Gambar 4.7 Star schema katalog web novel beserta jumlah baris setiap tabel

**Pseudocode 4.5 Pembangunan dimensi, bridge, dan fakta**

```
// source: src/phases/data_model.py
FUNCTION surrogate_key(df, natural_key, key_name):
    RETURN df + row_number() OVER (ORDER BY n_novels DESC, natural_key ASC) AS key_name
FUNCTION explode_dimension(novel, array_col, value_name, key_name):
    pairs <- DISTINCT(series_id, explode(array_col)) WHERE value IS NOT EMPTY
    dim <- surrogate_key(GROUP pairs BY value: n_novels = COUNT, value_name, key_name)
    bridge <- pairs JOIN dim ON value -> (series_id, key_name)
    RETURN dim, bridge
dim_genre, bridge_genre <- explode_dimension(novel, genre_list, "genre", "genre_key")
dim_tag, bridge_tag     <- explode_dimension(novel, tag_list, "tag", "tag_key")
dim_author, bridge_author <- resolve_aliases_then_explode(novel.authors)   // Pseudocode 3.5
dim_publisher <- surrogate_key(GROUP novel BY original_publisher)
```

```
dim_time <- DISTINCT(year, decade) WHERE year IS NOT NULL                 // valid years only
fact_novel <- novel JOIN dim_publisher -> (series_id, publisher_key, year, decade,
              additive measures, non-additive measures, text measures, flags)  // 1 row per
title
```

Ukuran di tabel fakta dibedakan menurut sifat agregasinya. `chapter_count`, `rating_votes`, dan `reading_list_count` bersifat aditif, sehingga boleh dijumlahkan. `weighted_rating`, `rating_score`, dan `days_per_chapter` bersifat non-aditif: menjumlahkan rating lintas judul tidak bermakna, jadi yang sah hanya rata-rata atau median.

**b. Operasi OLAP di dashboard**

Halaman *Data Warehouse* di dashboard menjawab pertanyaan analis langsung di atas star schema (Gambar 4.8). Pengguna memilih dimensi baris, dimensi pembanding (opsional), dan ukuran. Setiap tampilan dijalankan sebagai operasi OLAP berikut, dan dashboard menyebutkan operasi yang sedang aktif:

- **Slice:** satu dimensi dikunci pada satu nilai, misalnya hanya judul asal Tiongkok.
- **Dice:** beberapa dimensi dikunci sekaligus, misalnya asal CN, status tamat, dan tahun 2015 sampai 2026.
- **Roll-up dan drill-down:** hierarki waktu tahun ke dekade. Memilih dekade 2010 menampilkan rincian per tahun di dalamnya.
- **Pivot:** dua dimensi disilangkan menjadi matriks, misalnya genre dikali asal negara.

Logika agregasinya dirangkum pada Pseudocode 4.6. Dua aturan dijaga di sini. Ukuran non-aditif hanya ditampilkan untuk grup berisi minimal 10 judul, supaya tidak ada rata-rata yang bertumpu pada segelintir judul. Ukuran aditif yang dipecah menurut dimensi bernilai banyak diberi peringatan, karena jumlah semua batangnya melebihi total katalog.

**Pseudocode 4.6 Agregasi OLAP di halaman Data Warehouse**

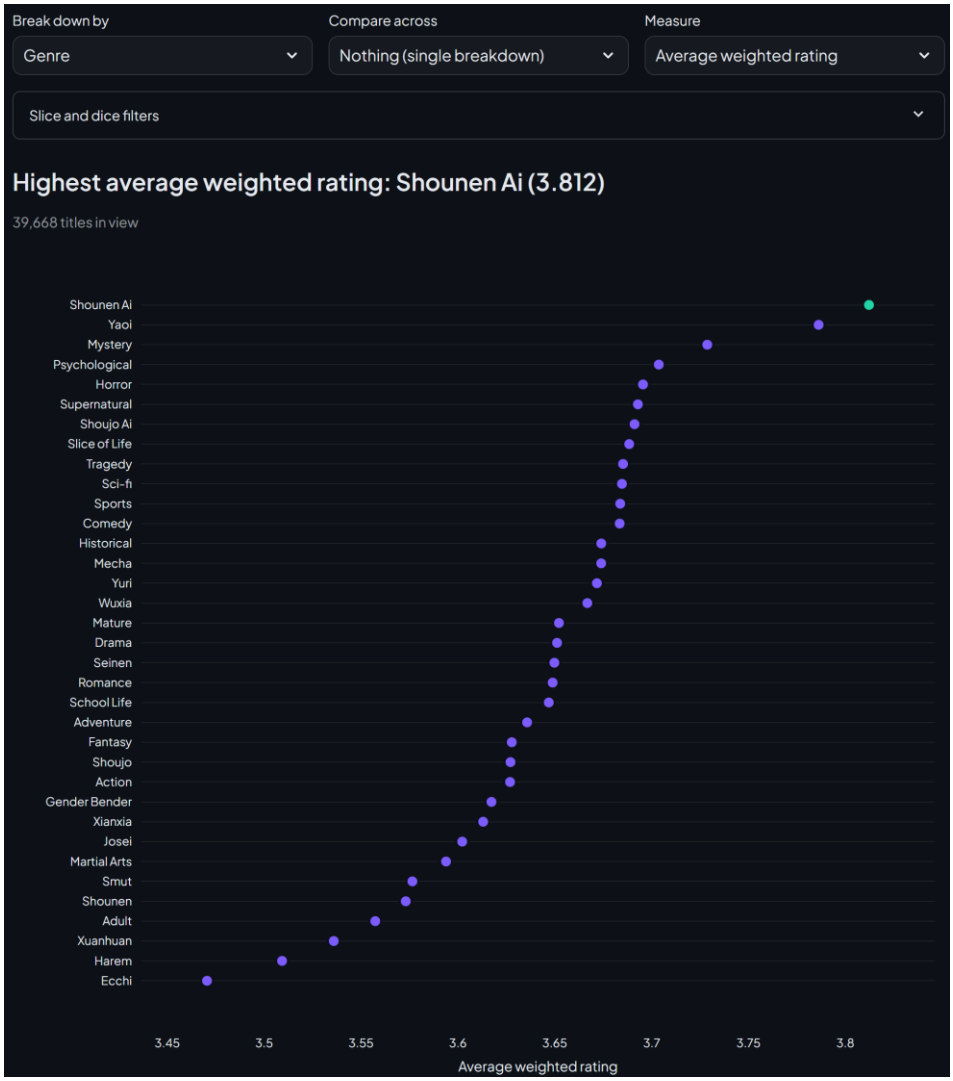
```
// source: app/streamlit_app.py
FUNCTION olap_view(row_dim, col_dim, measure, filters):
    facts <- fact_novel JOIN dim_novel JOIN dim_publisher        // one row per title
    facts <- apply filters on origin, format, status, year, >= 10 votes   // slice or dice
    frame <- (series_id, row_dim value)                          // exploded via bridge if
multi-valued
    IF col_dim EXISTS: frame <- frame JOIN (series_id, col_dim value)       // pivot
    result <- GROUP frame JOIN facts BY dimensions:
                  aggregate(measure), n_titles = COUNT_DISTINCT(series_id)
    IF measure is non-additive: result <- FILTER result WHERE COUNT(measure values) >= 10
    IF row_dim = "Decade" AND a decade is selected:              // drill-down
        show the same aggregation per Year within that decade
    IF measure is additive AND a multi-valued dimension is used:
        warn("bar total > catalogue total; read as reach per group")
    show(chart(result), equivalent_sql(row_dim, col_dim, measure, filters))
```

Dashboard juga menampilkan SQL setara untuk setiap tampilan. Contoh berikut adalah SQL untuk pivot genre dikali asal negara dengan ukuran rata-rata weighted rating pada judul tamat yang bersuara minimal 10:

```
SELECT   g.genre, n.type_origin,
         ROUND(AVG(f.weighted_rating), 3) AS measure,
         COUNT(DISTINCT f.series_id)      AS n_titles
FROM     fact_novel f
JOIN     dim_novel n           ON n.series_id = f.series_id
JOIN     bridge_novel_genre bg ON bg.series_id = f.series_id
JOIN     dim_genre g           ON g.genre_key = bg.genre_key
WHERE    n.is_completed = 1 AND f.is_rated_reliable
  AND    n.type_origin IS NOT NULL
GROUP BY g.genre, n.type_origin
HAVING   COUNT(f.weighted_rating) >= 10
ORDER BY measure DESC
```

Kebenaran SQL ini diuji dengan menjalankannya di Spark dan membandingkan hasilnya dengan perhitungan dashboard. Pada lima kombinasi dimensi, ukuran, dan filter yang berbeda, seluruh grup cocok dengan jumlah judul identik; selisih nilai hanya berasal dari pembulatan

ROUND. Selain itu, 588 kombinasi dimensi dan ukuran dijalankan tanpa error, dan jumlah judul per genre sama persis dengan kolom n_novels di dim_genre. Ketika ukuran aditif dipecah menurut genre, jumlah semua batang (misalnya 13.420.148 suara) melebihi total suara katalog (2.258.882), karena satu judul terhitung sekali di setiap genrenya.

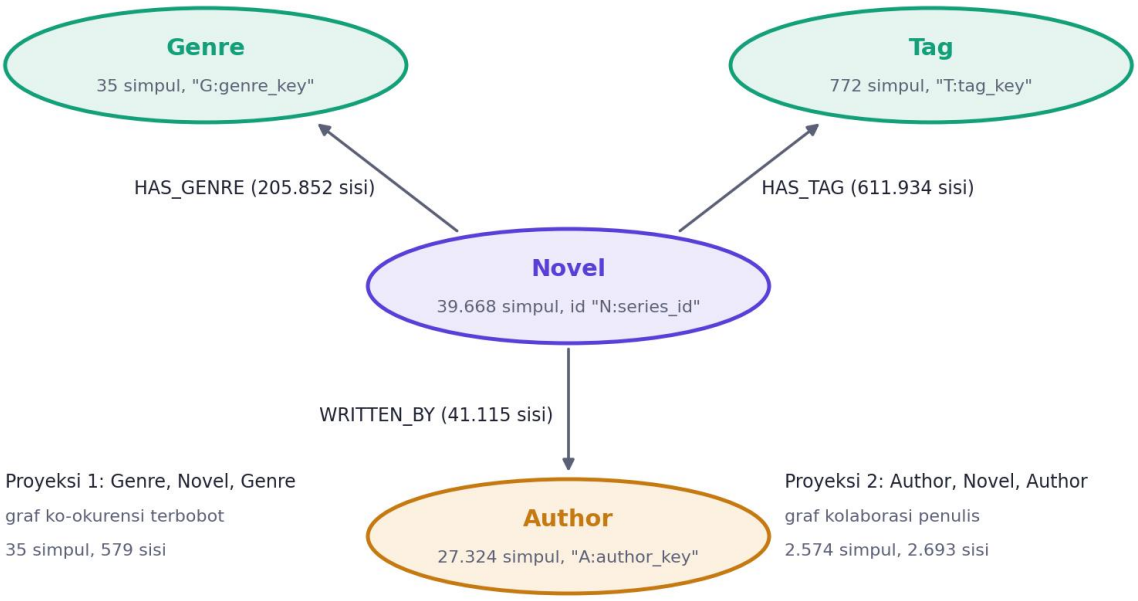



Gambar 4.8 Dashboard rata-rata weighted rating per genre, dengan genre tertinggi ditandai

Contoh hasil operasi pivot genre dikali asal negara (dimensi Genre dibandingkan dengan Country of origin di dashboard): di antara judul tamat dengan rating tepercaya, judul asal Tiongkok memiliki rata-rata tertinggi pada 11 dari 12 genre terbesar, misalnya Yaoi (3,93) dan Slice of Life (3,88). Satu-satunya pengecualian adalah Harem, tempat judul Korea unggul (3,61 dibanding 3,52 untuk judul Tiongkok). Temuan seperti ini langsung relevan bagi tim lisensi yang memilih judul per segmen.

### c. Property graph

Relasi antarentitas dimodelkan sebagai property graph dengan empat label simpul dan tiga tipe sisi (Gambar 4.9): 67.799 simpul dan 858.901 sisi. Setiap simpul diberi awalan tipe (N:, G:, T:, A:) supaya kunci genre 7 dan kunci tag 7 tidak bertabrakan menjadi satu simpul.




Gambar 4.9 Skema property graph dan dua proyeksi yang dianalisis

Dari graf ini dibuat dua proyeksi yang masing-masing menjawab pertanyaan berbeda. Proyeksi genre dibangun dengan self-join tabel `bridge_novel_genre` di Spark: dua genre dihubungkan bila minimal 5 judul menyandang keduanya, dan bobot sisinya adalah jumlah judul bersama. Proyeksi penulis hanya memakai judul dengan dua penulis atau lebih, sehingga sisinya adalah kolaborasi sungguhan, bukan sekadar kemiripan genre. Kedua graf lalu dianalisis dengan NetworkX seperti pada Pseudocode 4.7: PageRank (Page et al., 1999), sentralitas, komponen terhubung, dan deteksi komunitas greedy modularity (Clauset et al., 2004). Betweenness dihitung dengan sampel 200 simpul sumber karena perhitungan eksaknya berkompleksitas O(V x E).

**Pseudocode 4.7 Proyeksi dan analisis graf**

```
// source: src/phases/graph_analysis.py
// projection in Spark (distributed)
pairs <- bridge_novel_genre a JOIN bridge_novel_genre b
         ON a.series_id = b.series_id AND a.genre_key < b.genre_key
genre_edges <- GROUP pairs BY (a.genre_key, b.genre_key): weight = COUNT
               WHERE weight >= 5
multi <- titles in bridge_novel_author with >= 2 authors
author_edges <- DISTINCT author pairs on the same title IN multi
// in-memory analysis (NetworkX)
FOR EACH G IN {genre_graph(genre_edges), author_graph(author_edges)}:
    pr <- pagerank(G, weight)                          // influence from structure
    dc <- degree_centrality(G)
    bc <- betweenness_centrality(G, k = MIN(200, |V|), seed = 42)
    components <- connected_components(G)
    communities <- greedy_modularity_communities(G, weight)
    Q <- modularity(G, communities)
    save(pr, dc, bc, community_id) TO analytics/graph_*_pagerank
// traversal: find two non-adjacent genres linked through one intermediary
path <- shortest_path(genre_graph, g1, g2)   // proves depth <= 2 hops
```

Hasil analisis kedua proyeksi dirangkum pada Tabel 4.6.

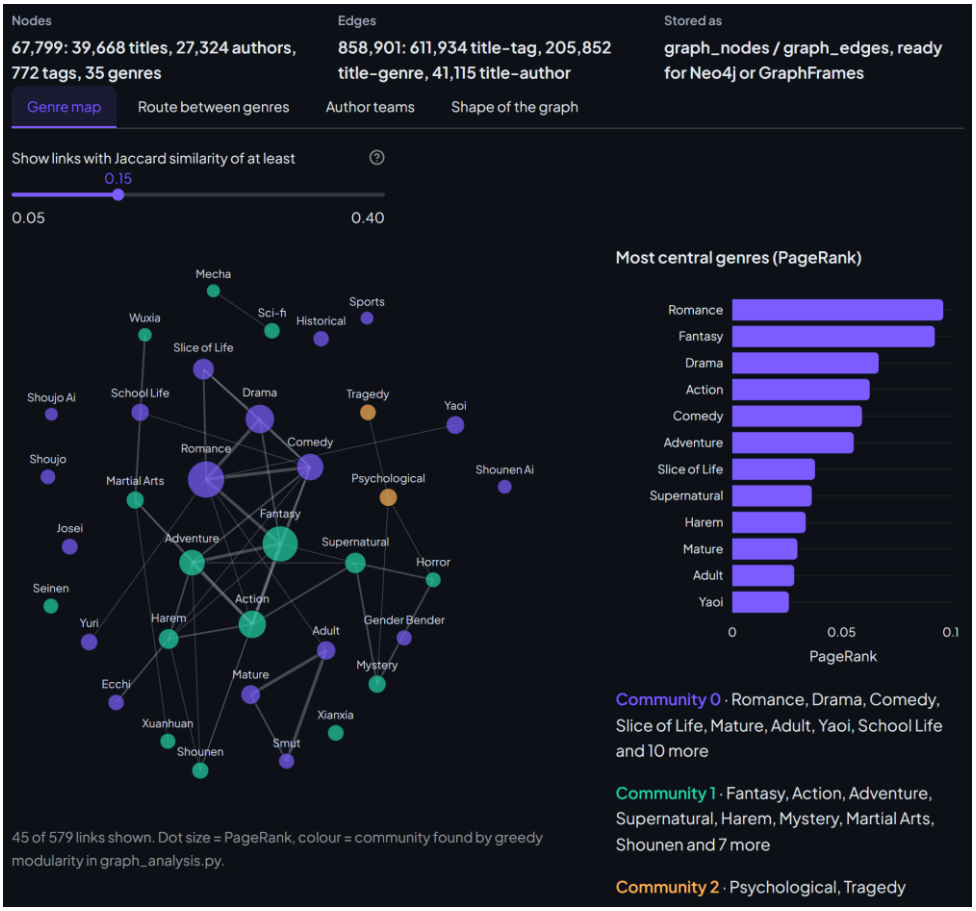
Tabel 4.6 Hasil analisis dua proyeksi graf

| Ukuran | Ko-okurensi genre | Kolaborasi penulis |
|---|---|---|
| Simpul / sisi | 35 / 579 | 2.574 / 2.693 |
| Komponen terhubung | 1 | 930 |
| Komponen terbesar | 35 simpul | 22 penulis |

| Ukuran | Ko-okurensi genre | Kolaborasi penulis |
|---|---|---|
| Komunitas (greedy modularity) | 3 | 930 |
| Modularity | 0,0858 | 0,9947 |
| Tafsiran | Genre saling berbaur, label genre saja lemah untuk membedakan judul | Tim penulis hampir tidak pernah berkolaborasi lintas tim |

Pada graf genre, PageRank tertinggi dimiliki Romance (0,0966) dan Fantasy (0,0927). Keduanya sering muncul dan terhubung dengan genre lain yang juga berpengaruh; PageRank memperhitungkan kedua hal itu. Kedekatan antar-genre diukur dengan koefisien Jaccard (jumlah judul bersama dibagi jumlah judul yang menyandang salah satunya), bukan jumlah mentah, supaya genre besar tidak otomatis tampak dekat dengan semua genre lain. Pasangan terdekat adalah Action dan Adventure (Jaccard 0,518), lalu Romance dan Drama (0,426). Modularity graf genre yang rendah (0,0858) sejalan dengan temuan EDA bahwa satu judul rata-rata menyandang lima genre: genre tidak membentuk kelompok yang tegas.

Halaman *Relationship Graph* di dashboard menampilkan peta genre berwarna komunitas dengan ambang Jaccard yang bisa diatur, serta pencari rute antar-genre (Gambar 4.10). Pada ambang 0,15, rute dari Yaoi ke Martial Arts membutuhkan tiga hop (Yaoi, Romance, Action, Martial Arts). Tanpa ambang, rute terpanjang dalam graf (diameter) hanya dua hop: dari 595 pasangan genre, hanya 16 yang tidak bertetangga langsung, dan semuanya terhubung lewat satu genre perantara, misalnya Yaoi ke Yuri. Pengukuran ini membuktikan klaim kedalaman dua hop yang menjadi alasan tidak memakai graph database terpisah (Subbab 2.2).




Gambar 4.10 Peta genre dan peringkat PageRank

**d. Model untuk data semi-terstruktur dan tidak terstruktur**

Untuk jalur MapReduce, data semi-terstruktur disimpan sebagai dokumen JSONL, satu objek per judul dengan genre, tag, dan penulis sebagai array (Pseudocode 4.3). Mapper cukup mengiterasi array tersebut, tanpa perlu mengurai string berkoma. Sinopsis diubah menjadi matriks fitur TF-IDF untuk menangkap frasa khas seperti "cultivation", lalu dipadatkan dengan SVD agar tahan terhadap sinonim. Pilihan TF-IDF didukung temuan EDA bahwa kata tersering di korpus bersifat generik ("one", "world", "life"), sehingga perlu dibobot turun. Matriks inilah yang dipakai untuk kemiripan kosinus pada rekomendasi dan klasifikasi genre multilabel. Evaluasi model tersebut dibahas pada Bab 5.

## 4.3 Dataset untuk Diolah Menggunakan *Machine Learning*

Dari variasi data yang diulas pada subbab 4.1, data yang dipilih untuk diolah lebih lanjut adalah tabel `novels_clean` di zona clean, yaitu katalog hasil pembersihan Fase 2. Tabel ini dipilih karena memuat ketiga kelas format sekaligus untuk setiap judul: sinopsis sebagai teks tidak terstruktur, genre dan tag sebagai array semi-terstruktur, serta asal negara, tipe, jumlah bab, tahun, status, dan rating sebagai kolom terstruktur. Judul yang ikut dimodelkan hanya yang punya sinyal teks, yaitu minimal tiga token bersih setelah judul, sinopsis, dan tag digabung. Syarat ini dipenuhi 39.660 dari 39.668 judul.

**a. Masalah yang diselesaikan**

Masalah utamanya adalah **penandaan genre otomatis**. Bab 1 mencatat bahwa label genre bersifat subjektif: satu judul rata-rata menyandang 5,19 genre, dan judul baru sering masuk katalog dengan genre yang belum lengkap. Tim lisensi dan tim kurasi membutuhkan label yang konsisten untuk memfilter kandidat. Karena itu model diminta memprediksi himpunan genre sebuah judul dari sinopsis, tag, dan metadatanya. Ini klasifikasi *multi-label*: setiap judul bisa benar di beberapa genre sekaligus, dan 35 genre yang dipakai sangat timpang. Romance muncul pada 63,2% judul, sedangkan Shoujo Ai hanya pada 690 judul (1,7%).

Tiga masalah pendukung dikerjakan di atas representasi teks yang sama. Pertama, rekomendasi berbasis konten: judul mana yang mirip dengan judul yang sudah sukses, tanpa data perilaku pembaca karena dataset ini tidak memiliki `user_id`. Kedua, segmentasi katalog menjadi arketipe cerita untuk perencanaan konten. Ketiga, estimasi rating untuk judul yang belum cukup dinilai, sebab 41,4% judul punya kurang dari 10 suara. Keempatnya disebut Head A sampai Head D pada sisa laporan ini.

**b. Elemen data yang dipakai**

Tabel 4.7 memetakan kolom yang masuk ke model beserta perannya. Dua kolom sengaja dikeluarkan dari fitur Head D, yaitu `rating_votes` dan `reading_list_count`. Keduanya baru terisi setelah pembaca menilai dan menyimpan sebuah judul, dan EDA pada subbab 4.1.4 menunjukkan keduanya berkorelasi dengan weighted rating (0,43 dan 0,44) serta satu sama lain (0,91). Memakai keduanya sebagai fitur berarti membocorkan sinyal popularitas yang tidak tersedia untuk judul baru, padahal judul baru itulah sasaran Head D. Jumlah suara hanya dipakai sebagai bobot sampel saat Head D dilatih (subbab 5.3), sehingga judul baru yang belum punya suara tetap bisa diestimasi.

Tabel 4.7 Elemen data yang dipakai pada tahap *machine learning*

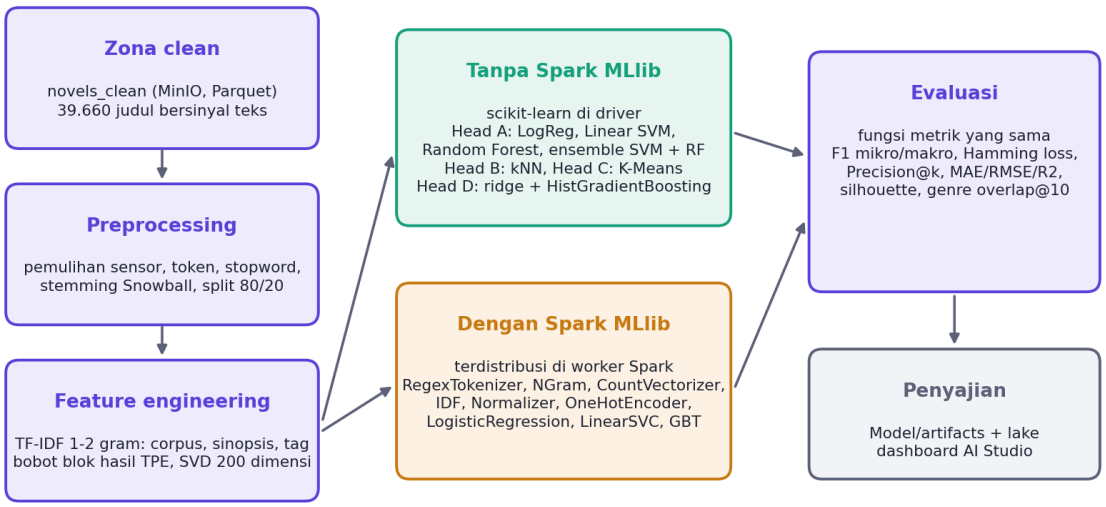
| Kolom | Kelas format | Peran | Keterangan |
|---|---|---|---|
| `corpus_clean` | Tidak terstruktur | Fitur Head A, B, C, D | Judul + sinopsis + tag, dibangun ulang di Fase 4 dengan kata bersensor dipulihkan |
| `synopsis_clean` | Tidak terstruktur | Fitur Head A | Sinopsis saja, diberi ruang IDF sendiri |
| `tags` | Semi-terstruktur | Fitur Head A, D | 772 tag berbeda, rata-rata 15,43 per judul |

| Kolom | Kelas format | Peran | Keterangan |
|---|---|---|---|
| genre_list | Semi-terstruktur | Label Head A | 35 genre, rata-rata 5,19 per judul |
| type_origin, type | Terstruktur | Fitur Head A, D | One-hot; asal negara sangat menentukan genre seperti Xianxia |
| chapter_count, year, is_completed, days_per_chapter, n_tags, token_count | Terstruktur | Fitur Head D | Log untuk kolom menceng, imputasi median |
| rating_score | Terstruktur | Target Head D | Hanya judul dengan >= 10 suara (23.256 judul) |
| rating_votes, reading_list_count | Terstruktur | Bukan fitur | Membocorkan target Head D; rating_votes hanya dipakai sebagai bobot sampel latih |

Pembagian data dibuat sekali dan dipakai semua eksperimen: 31.728 judul (80%) untuk pengembangan dan 7.932 judul (20%) untuk pengujian. Untuk Head D, populasi tepercaya (23.256 judul, sama dengan populasi EDA) dibagi dengan aturan yang sama menjadi 18.604 judul latih dan 4.652 judul uji.

# BAB 5
# MACHINE LEARNING UNTUK BIG DATA

Pipeline *machine learning* disusun di atas data lake yang sama dengan Assignment I (Gambar 5.1). Data dibaca dari zona clean oleh Spark, diproses melalui tahap preprocessing dan feature engineering, lalu dilatih melalui dua jalur: tanpa Spark MLlib (scikit-learn di driver, Fase 4) dan dengan Spark MLlib (terdistribusi di worker, Fase 5). Kedua jalur dievaluasi dengan fungsi metrik yang sama. Seluruh pipeline ditulis dalam Python; kodenya berada di `src/phases/phase4_model.py`, paket `Model/pipeline/`, dan `src/phases/phase5_mllib.py`.




Gambar 5.1 Pipeline *machine learning* dari zona clean sampai penyajian

## 5.1 *Data Preprocessing*

### a. Normalisasi dan tokenisasi teks

Preprocessing teks sudah dimulai di Fase 2 dengan Spark ML. Judul, sinopsis, dan tag digabung menjadi satu korpus, diubah ke huruf kecil, lalu semua karakter selain huruf Latin diganti spasi. Angka, tanda baca, dan aksara CJK ikut terbuang. `RegexTokenizer` memecah teks menjadi token minimal tiga huruf, dan `StopWordsRemover` membuang stopword bahasa Inggris bawaan Spark ditambah stopword domain seperti "novel", "story", dan "chapter" yang muncul di hampir semua sinopsis. Hasilnya disimpan sebagai `corpus_clean` dan `synopsis_clean`.

### b. Pemulihan kata bersensor

NovelUpdates menyensor sebagian kosakata dengan tanda bintang, satu bintang untuk setiap huruf yang disembunyikan, misalnya `mu*der`, `r*pe`, dan `S*aves`. Sekitar 15% sinopsis dan 15% daftar tag terkena sensor ini. Regex Fase 2 mengganti bintang dengan spasi, sehingga `mu*der` pecah menjadi "mu" dan "der" dan kata aslinya hilang dari fitur. Karena itu Fase 4 membangun ulang `corpus_clean` dan `synopsis_clean` dari teks mentah setelah setiap bentuk bersensor dicocokkan ke daftar kata sasaran yang diurutkan menurut prioritas. Daftar tetap dipakai, bukan pencocokan otomatis ke kosakata korpus, karena situs juga menyensor kata yang hanya memuat potongan terlarang (`gr*pe` untuk grape) dan pencocokan otomatis memetakan `r*pe` ke "rope". Bentuk yang tidak cocok dengan daftar cukup dibuang bintangnya. Fungsi yang sama dipanggil saat inferensi di dashboard, sehingga teks yang diketik pengguna diproses persis seperti data latih.

**c. Stemming**

Sebelum divektorkan, setiap token di-*stem* dengan Snowball. Tanpa langkah ini "cultivation", "cultivating", dan "cultivator" menjadi tiga fitur terpisah yang masing-masing lebih jarang muncul. Stemming dikerjakan di dalam pipeline scikit-learn sebagai fungsi tingkat modul, sehingga ikut tersimpan di artefak dan dijalankan ulang secara identik saat inferensi di dashboard.

**d. Matriks label dan pembagian data**

Daftar genre diubah menjadi matriks biner 39.660 x 35 dengan `MultiLabelBinarizer`. Genre dengan kurang dari 50 judul seharusnya dibuang karena klasifikatornya hanya akan selalu menebak "bukan". Pada katalog 39k, genre terkecil (Mecha dan Shoujo Ai) masih punya sekitar 690 judul, jadi tidak ada yang terbuang. Sebelum dibagi, baris diurutkan menurut `series_id`. Urutan hasil `toPandas()` bergantung pada partisi Spark dan bisa berbeda antar eksekusi; tanpa pengurutan ini skor makro-F1 bergeser sekitar 0,003 walau kode model tidak berubah.

**e. Nilai kosong pada fitur numerik**

Head D hanya dilatih pada judul dengan minimal 10 suara, karena rating dari satu atau dua suara lebih banyak derau daripada sinyal. Jumlah bab, tahun, dan hari per bab yang kosong diisi median, lalu jumlah bab dan hari per bab diubah ke skala log karena distribusinya sangat menceng (subbab 4.1.4).

**Pseudocode 5.1 Preprocessing teks, label, dan pembagian data**

```
// source: phase2_clean.py, text_prep.py, pipeline/preprocessing.py, phase4_model.py
corpus_raw  <- concat(title, synopsis, tags)
corpus_raw  <- uncensor(lower(corpus_raw))                    // Phase 4: mu*der -> murder
corpus_norm <- regexp_replace(lower(corpus_raw), "[^a-z\s]", " ")
tokens      <- RegexTokenizer(corpus_norm, pattern = "\s+", minTokenLength = 3)
tokens      <- StopWordsRemover(tokens, english + {novel, story, chapter, ...})
corpus_clean <- join(tokens, " ")
data   <- FILTER novels_clean WHERE token_count >= 3          // 39,660 of 39,668
data   <- SORT data BY series_id                              // toPandas order is not stable
labels <- genres with >= 50 titles                            // 35 of 35 kept
Y      <- MultiLabelBinarizer(labels).transform(genre_list)   // 39,660 x 35
dev, test <- train_test_split(rows, test_size = 0.2, seed = 42)
stem(doc) <- Snowball stem of each token, memoized             // cultivating -> cultiv
```

## 5.2 *Feature Engineering*

Ruang fitur Head A disusun dari tiga pandangan teks dan satu blok metadata (Tabel 5.1). Ketiga blok teks sengaja tumpang tindih. Pada eksperimen awal dengan 4.460 judul, blok tag saja menghasilkan makro-F1 0,507, sinopsis saja 0,455, dan gabungan ketiganya 0,547. Sinopsis diberi ruang IDF sendiri agar kosakata cerita tidak tertimbun frasa tag yang jauh lebih seragam.

Tabel 5.1 Blok fitur pada pipeline *machine learning*

| Blok | Sumber | Transformasi | Dimensi maks. | Bobot |
|---|---|---|---|---|
| Corpus | `corpus_clean` | Stemming, TF-IDF 1-2 gram, min_df 3, max_df 0,85, sublinear tf | 75.000 | 2,31 |
| Sinopsis | `synopsis_clean` | Sama dengan corpus | 75.000 | 0,95 |
| Tag | `tags` | Koma jadi spasi, sensor dipulihkan, stemming, TF-IDF 1-2 gram, min_df 2 | 15.000 | 1,48 |
| Metadata | `type_origin`, `type` | One-hot, kategori < 20 judul digabung | < 20 | 0,48 |
| Representasi bersama | `corpus_clean` | TF-IDF 30.000 fitur, Truncated SVD 200 komponen, normalisasi L2 | 200 | - |
| Numerik Head D | 6 kolom terstruktur | log1p, imputasi median, one-hot asal dan tipe | 20 | - |

| Blok | Sumber | Transformasi | Dimensi maks. | Bobot |
|---|---|---|---|---|
| Tag Head D | `tags` | Multi-hot tag yang muncul di >= 50 judul | 678 | - |
| Skor teks Head D | `corpus_clean`, `tags` | Ridge pada TF-IDF, prediksi out-of-fold 5-fold (stacking) | 1 | - |

**a. TF-IDF n-gram**

TF-IDF (Salton & Buckley, 1988) memberi bobot tinggi pada frasa yang khas untuk sedikit judul dan menurunkan kata umum. Bigram dipakai karena banyak tag berbentuk frasa, misalnya "time loop" atau "weak to strong", yang maknanya hilang bila dipecah menjadi unigram. `sublinear_tf` meredam kata yang berulang di satu dokumen. Bobot keempat blok (corpus 2,31, sinopsis 0,95, tag 1,48, metadata 0,48) dicari bersama parameter C Linear SVM dengan TPE (subbab 5.4). Bobot sebelumnya (1,0, 1,0, 1,6, dan 0,6) dipilih saat katalog baru berisi 4.460 judul dan belum pernah disetel ulang. Batas kosakata dinaikkan dari 40.000 menjadi 75.000 untuk corpus dan sinopsis, dan dari 10.000 menjadi 15.000 untuk tag. Kosakata yang lebih besar lagi (150.000) hanya menambah F1 makro 0,0008 pada uji lima pembagian data, dengan jumlah fitur dua kali lipat.

**b. Representasi bersama dengan SVD**

Head B, C, dan D memakai satu representasi padat. Matriks TF-IDF 30.000 fitur dipadatkan dengan Truncated SVD (*latent semantic analysis*, Deerwester et al., 1990) menjadi 200 komponen yang menjelaskan 15,31% variansi, lalu dinormalisasi L2 sehingga hasil kali titik sama dengan cosine similarity. Representasi ini juga menjadi dasar graf kemiripan di Bab 6.

**c. Fitur numerik dan pencegahan kebocoran**

Head D memakai 200 komponen SVD, 20 fitur numerik (log jumlah bab, jumlah tag, jumlah token, tahun, status tamat, log hari per bab, dan one-hot asal serta tipe), 678 tag dalam bentuk multi-hot, dan satu skor teks. Skor teks adalah prediksi rating dari ridge regression pada TF-IDF judul, sinopsis, dan tag. Untuk baris latih, skor ini diambil dari prediksi *out-of-fold* 5-fold (*stacked generalization*, Wolpert, 1992), sehingga HistGradientBoosting tidak pernah melihat skor yang dihitung dari rating baris itu sendiri. Skor ini mengembalikan informasi frasa jarang yang hilang ketika teks dipadatkan ke SVD 200 dimensi. Kecepatan rilis dimasukkan karena merupakan keputusan penulis atau penerjemah, bukan turunan jumlah suara, sehingga tidak membocorkan target.

**d. Padanan fitur di Spark MLlib**

Jalur MLlib membangun tiga blok teks yang sama (corpus, sinopsis, tag) dengan komponen bawaan: `RegexTokenizer`, `NGram`, `SQLTransformer` untuk menggabungkan unigram dan bigram, `CountVectorizer`, `IDF`, dan `Normalizer`, ditambah `StringIndexer` dan `OneHotEncoder` untuk metadata. Hasilnya 90.016 dimensi. `Normalizer` ternyata wajib. `TfidfVectorizer` menormalisasi setiap dokumen ke panjang L2 = 1, sedangkan `IDF` di Spark tidak. Pada uji coba pertama tanpa normalisasi, Logistic Regression MLlib mencapai log loss latih 0,0007 tetapi log loss uji 0,97: model menghafal data latih karena norma vektor yang besar membuat regularisasi yang setara di atas kertas menjadi terlalu lemah. Tiga hal tetap tidak punya padanan bawaan, yaitu stemming Snowball, `sublinear_tf`, dan bobot per blok (tag 1,6, metadata 0,6). Ketiganya sengaja tidak ditambal dengan UDF Python supaya pipeline tetap murni MLlib.

Sisi scikit-learn pada perbandingan ini sengaja dikunci pada konfigurasi Fase 4 yang pertama: kosakata 40.000, 40.000, dan 10.000 fitur, bobot blok 1,0, 1,0, 1,6, dan 0,6, teks Fase 2 tanpa pemulihan sensor, serta nilai C dari pemenang awal. Dengan begitu ukuran blok sama

dengan blok MLlib, dan angka Tabel 5.3 tetap dapat diulang walaupun konfigurasi produksi di Tabel 5.1 sudah disetel ulang.

## 5.3 Algoritma *Machine Learning*

Algoritma dipilih per tugas, dengan mempertimbangkan bentuk data (teks berdimensi tinggi dan *sparse*) serta ketersediaan versi yang dapat diskalakan (Tabel 5.2).

Tabel 5.2 Algoritma per tugas dan padanannya yang dapat diskalakan

| Tugas | scikit-learn (tanpa MLlib) | Spark MLlib | Alasan |
|---|---|---|---|
| Head A: genre multi-label | One-vs-Rest Logistic Regression; One-vs-Rest Linear SVM + kalibrasi Platt; Random Forest multi-output; ensemble log-odds SVM + Random Forest | `LogisticRegression` dan `LinearSVC` per genre; random forest per genre (opsi) | Model linear unggul pada teks sparse berdimensi tinggi dan cepat dilatih (Fan et al., 2008); pohon menangkap interaksi kata yang tidak terlihat model linear (Breiman, 2001) |
| Head B: rekomendasi | kNN cosine pada SVD 200 dimensi | `BucketedRandomProjectionLSH` (opsi) | Tidak ada data perilaku, jadi rekomendasi berbasis konten |
| Head C: arketipe | K-Means, k dipilih dengan silhouette | `KMeans` | Segmen yang bisa dibaca kurator dari istilah dominan |
| Head D: rating | Ridge (skor teks) + HistGradientBoostingRegressor | `GBTRegressor` | Hubungan fitur dengan rating tidak linear (subbab 4.1.4) |

**a. Logistic Regression dan Linear SVM untuk multi-label**

Klasifikasi multi-label dipecah menjadi 35 masalah biner (*binary relevance*, Zhang & Zhou, 2014): satu klasifikator per genre. Logistic Regression menghasilkan probabilitas yang terkalibrasi. Linear SVM memaksimalkan margin dan biasanya lebih tajam pada teks, tetapi hanya menghasilkan jarak ke *hyperplane*, sehingga jarak itu diubah menjadi probabilitas dengan kalibrasi sigmoid (Platt, 1999). Probabilitas dibutuhkan karena dashboard menampilkan genre terurut menurut keyakinan model.

**b. Ambang keputusan per genre**

Ambang 0,5 yang seragam mengasumsikan semua genre sama seringnya. Kenyataannya probabilitas Mecha hampir tidak pernah menyentuh 0,5. Karena itu setiap genre mendapat ambangnya sendiri, dicari pada rentang 0,05 sampai 0,80 dengan langkah 0,025 untuk memaksimalkan F1 genre tersebut (Fan & Lin, 2007). Ambang dicari dari probabilitas *out-of-fold*, bukan dari set uji, agar skor uji tidak bias.

**c. Random Forest dan ensemble**

Random Forest (Breiman, 2001) ditambahkan sebagai model berbasis pohon. Satu hutan multi-output memprediksi ke-35 genre sekaligus. Madjarov et al. (2012) membandingkan 12 metode multi-label pada 11 dataset dan mendapati *random forest of predictive clustering trees* termasuk yang terbaik. Pohon tidak efisien pada ratusan ribu kolom TF-IDF yang jarang terisi, sehingga masukannya disaring menjadi 3.000 fitur dengan skor chi-kuadrat tertinggi untuk genre mana pun. Bobot kelas tidak dipakai: pada hutan multi-output, bobot 35 genre dikalikan per sampel sehingga menjadi ekstrem, sedangkan ketimpangan kelas sudah ditangani ambang per genre. Hutan dibatasi 60 pohon dengan maksimum 2.000 daun (sekitar 143 MB di memori). Hutan 150 pohon x 3.000 daun butuh sekitar 535 MB, tetapi pada uji ukuran hutan skor OOF ensemble-nya hanya 0,0004 lebih tinggi.

Ensemble menggabungkan Linear SVM dan Random Forest dengan merata-ratakan *log-odds* probabilitas keduanya (*logarithmic opinion pool*, Genest et al., 1984), lalu hasilnya dikembalikan ke probabilitas dengan fungsi sigmoid. Bobotnya (SVM 0,7, Random Forest 0,3) dipilih pada grid 0 sampai 1 atas probabilitas *out-of-fold*, sama seperti hyperparameter lain, dan ambang per genre disetel ulang untuk probabilitas gabungan. Tidak ada model yang dilatih ulang: ensemble hanya menyimpan bobot dan ambangnya. Rata-rata probabilitas biasa sempat diuji lebih dulu. F1 makronya setara, tetapi Precision@3 turun 0,0040 dibanding SVM saja pada lima pembagian data ($p = 0,02$), sedangkan rata-rata log-odds hanya menurunkannya 0,0014 dan tidak bermakna ($p = 0,14$).

**d. K-Means dan Gradient Boosting**

K-Means dijalankan pada $k = 5$ sampai 10, dan k dipilih dengan *silhouette score* (Rousseeuw, 1987) pada subsampel 2.000 titik. Untuk rating, HistGradientBoostingRegressor (varian *gradient boosting*, Friedman, 2001) dilatih di atas fitur Head D, termasuk skor ridge dari subbab 5.2. *Hyperparameter*-nya dicari dengan TPE (25 percobaan, validasi silang 3-fold pada data latih): laju belajar 0,0212, maksimum 33 daun per pohon, minimal 43 sampel per daun, regularisasi L2 0,00144, 20,4% fitur per pemisahan, dan *early stopping* dengan kesabaran 20 iterasi; pelatihan berhenti di iterasi ke-363. Setiap judul diberi bobot $\log(1 + \text{jumlah suara})$, karena rating dari ratusan suara jauh lebih stabil daripada rating dari 10 suara.

**e. Opsi yang dapat diskalakan**

Spark MLlib (Meng et al., 2016) dipilih sebagai jalur terdistribusi karena sudah menjadi bagian dari stack Assignment I: data dibaca langsung dari MinIO tanpa dipindah, dan model dilatih di worker Spark yang sama. TensorFlow on Big Data (misalnya TensorFlowOnSpark atau Horovod) juga dipertimbangkan, tetapi tidak dipakai. Masalah ini diselesaikan dengan model *machine learning* klasik di atas fitur TF-IDF, yang tidak membutuhkan pelatihan jaringan saraf terdistribusi, dan 39.660 dokumen masih muat di memori satu mesin.

## 5.4 Proses Pengembangan Model dengan dan tanpa Spark MLlib

**a. Tanpa Spark MLlib (Fase 4)**

Spark tetap dipakai untuk bagian yang berat di sisi data: membaca Parquet dari zona clean, menyaring judul bersinyal teks, dan memilih 26 kolom. Hasilnya (39.660 baris, sekitar 60 MB) di-*collect* ke driver dan dilatih dengan scikit-learn (Pedregosa et al., 2011). Pemilihan *hyperparameter* memakai probabilitas *out-of-fold* 3-fold pada set pengembangan. Kandidat C dan bobot blok fitur berasal dari pencarian TPE (*Tree-structured Parzen Estimator*, Bergstra et al., 2011) dengan Optuna (Akiba et al., 2019): 40 percobaan untuk Linear SVM, dengan konfigurasi lama sebagai percobaan pertama. Sepuluh percobaan teratas hanya berselisih 0,0006 F1 makro, jadi optimumnya landai. Bobot kelas balanced kalah di semua percobaan teratas, karena ambang per genre sudah mengoreksi ketimpangan kelas. Konfigurasi terbaik dilatih ulang pada seluruh set pengembangan, koefisien model linear disimpan sebagai float32 (tidak ada keputusan genre yang berubah), lalu model diuji sekali pada set uji (Pseudocode 5.2). Model pemenang dipilih dari skor OOF, bukan skor uji, agar set uji tidak ikut menentukan pilihan.

**Pseudocode 5.2 Pelatihan Head A tanpa Spark MLlib**

```
// source: Model/pipeline/train.py
FOR model IN {logreg, svm, rf}:
    FOR params IN PARAM_GRID[model]:                // logreg: 2, svm: 2, rf: 1 configs
        P_oof <- cross_val_predict(pipeline(params), dev, Y_dev, KFold(3))
        thr   <- FOR EACH genre g: argmax_t F1(Y_dev[g], P_oof[g] >= t)
                 t IN 0.05 .. 0.80 step 0.025
        score <- macro_F1(Y_dev, P_oof >= thr)
    best  <- params with the highest score
    final <- pipeline(best).fit(dev)                // all 31,728 dev rows
    final.coef <- float32(final.coef)               // no genre decision changes
    report <- metrics(Y_test, final.predict_proba(test), thr_best)
P_ens(w) <- sigmoid(w * logit(P[svm]) + (1 - w) * logit(P[rf]))  // w in 0.0 .. 1.0
w, thr   <- best OOF macro_F1 over w; ensemble scored on test like the others
winner <- argmax over models of OOF score           // never the test score
save_atomic(final, thresholds, labels) -> Model/artifacts + curated-data/models/
```

### b. Dengan Spark MLlib (Fase 5)

Jalur ini melatih Head A dan Head D di klaster Spark Standalone proyek (satu worker, 2 core, memori executor 2 GB). `Pipeline` fitur di-*fit* hanya pada data latih, lalu hasil transformasinya di-*cache*. Untuk setiap genre, kolom label dibentuk dengan `array_contains`, kemudian `LogisticRegression` dan `LinearSVC` dilatih. Regularisasi disamakan dengan scikit-learn: scikit-learn meminimalkan C x jumlah loss + 1/2 x norma bobot, sedangkan Spark meminimalkan rata-rata loss + regParam/2 x norma bobot, sehingga regParam = 1 / (C x n latih). Bobot kelas *balanced* untuk SVM diberikan lewat `weightCol` (Pseudocode 5.3).

**Pseudocode 5.3 Pelatihan Head A dengan Spark MLlib**

```
// source: src/phases/phase5_mllib.py
block(col, vocab) <- [RegexTokenizer, NGram(2), SQLTransformer(concat uni + bi),
                      CountVectorizer(vocab, maxDF 0.85), IDF, Normalizer(L2)]
feat <- Pipeline(block(corpus, 40k) + block(synopsis, 40k) + block(tags, 10k)
                 + [StringIndexer, OneHotEncoder, VectorAssembler]).fit(X_fit)
X <- feat.transform(all).cache()
FOR g IN 35 genres:
    label <- array_contains(genre_list, g)
    lr  <- LogisticRegression(regParam = 1 / (4.0 * n_fit)).fit(X_fit)
    svc <- LinearSVC(regParam = 1 / (0.3 * n_fit), weightCol = balanced).fit(X_fit)
    history[g] <- model.summary.objectiveHistory          // loss per iteration
    p_fit, p_val, p_test <- score(model, X_fit | X_val | X_test)
    IF model = svc: p <- platt_sigmoid(fit on validation margins)
thr <- tuned on validation; metrics on the same 7,932 test rows as Phase 4
```

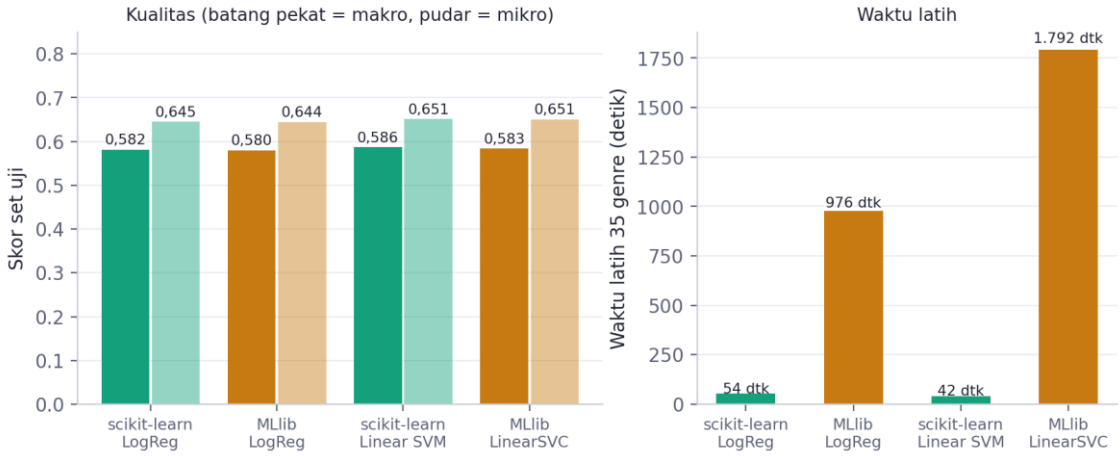
### c. Protokol pembanding yang adil

Melatih k-fold untuk 70 model MLlib (35 genre x 2 algoritma) terlalu mahal pada worker 2 core. Karena itu kedua kerangka diuji ulang dengan protokol yang sama: dari set pengembangan, 26.968 judul dipakai untuk melatih dan 4.760 judul (15%) untuk validasi ambang. Set uji berisi 7.932 judul yang dibentuk dengan aturan yang sama dengan Fase 4. Zona clean sudah diperbarui sejak Fase 4 dilatih (32 judul berganti dan 0,2% daftar genre berubah), sehingga scikit-learn ikut dilatih ulang pada snapshot yang sama dengan MLlib. Dengan begitu kedua kerangka diuji pada baris, genre, dan fungsi metrik yang identik. Untuk Head D, kedua kerangka diberi fitur tabular yang identik: fitur numerik Fase 4, multi-hot 35 genre, dan 150 tag tersering (205 fitur), karena embedding SVD dibangun di scikit-learn dan tidak tersedia di sisi MLlib.

### d. Hasil pembanding

Tabel 5.3 Head A dengan dan tanpa Spark MLlib (protokol validasi yang sama)

| Kerangka dan model | F1 makro | F1 mikro | Precision@3 | Hamming loss | Log loss uji | Waktu latih (dtk) |
|---|---|---|---|---|---|---|
| scikit-learn Logistic Regression | 0,5815 | 0,6450 | 0,7760 | 0,1175 | 0,2496 | 54 |
| Spark MLlib LogisticRegression | 0,5797 | 0,6438 | 0,7767 | 0,1178 | 0,2435 | 976 |
| scikit-learn Linear SVM | 0,5864 | 0,6510 | 0,7836 | 0,1142 | 0,2324 | 42 |
| Spark MLlib LinearSVC | 0,5834 | 0,6506 | 0,7834 | 0,1145 | 0,2319 | 1.792 |




Gambar 5.2 Kualitas dan waktu latih Head A dengan dan tanpa Spark MLlib

Pada kualitas, kedua kerangka praktis setara. F1 makro MLlib LogisticRegression 0,5797 berbanding 0,5815 di scikit-learn, dan MLlib LinearSVC 0,5834 berbanding 0,5864 (Tabel 5.3 dan Gambar 5.2). Selisih di bawah 0,004 ini lebih kecil daripada selisih antaralgoritma, dan dapat dijelaskan oleh tiga perbedaan fitur di subbab 5.2: MLlib tidak melakukan stemming, tidak memakai `sublinear_tf`, dan tidak memberi bobot berbeda pada blok tag. Linear SVM tetap unggul tipis atas Logistic Regression di kedua kerangka, sama seperti hasil Fase 4.

Pada waktu latih, perbedaannya besar. MLlib membutuhkan 16,3 menit untuk 35 model LogisticRegression (18 kali lebih lambat dari scikit-learn) dan 29,9 menit untuk LinearSVC (43 kali lebih lambat), belum termasuk 53 detik untuk membangun 90.016 fitur. Setiap pemanggilan `fit` dipecah menjadi puluhan job Spark, satu per iterasi optimisasi, dan setiap job membawa biaya penjadwalan, serialisasi, dan komunikasi driver-executor. Pada klaster satu worker dengan 2 core, biaya tetap ini jauh lebih besar daripada keuntungan paralelisme. scikit-learn sebaliknya menjalankan seluruh iterasi di satu proses dengan data di memori.

Head D memperlihatkan pola yang sama. Dengan 205 fitur yang identik dan 239 pohon, GBTRegressor MLlib mencapai MAE 0,3397 dalam 86 detik, sedangkan HistGradientBoosting mencapai MAE 0,3303 dalam 2,1 detik. Dua kendala teknis muncul saat pengembangan. Pertama, GBT MLlib dengan ratusan pohon gagal dengan `StackOverflowError` di executor karena lineage RDD terus memanjang; masalah ini selesai setelah direktori checkpoint dipasang di bucket curated-data. Kedua, *early stopping* MLlib tidak punya parameter kesabaran (*patience*), sehingga boosting berhenti pada kenaikan galat validasi pertama, yaitu di pohon ke-35. Karena itu jumlah pohon MLlib disamakan dengan pilihan *early stopping* HistGradientBoosting.

## 5.5 Evaluasi Model

Metrik Head A mengikuti konvensi klasifikasi multi-label. F1 makro merata-ratakan F1 tiap genre sehingga genre langka sama pentingnya dengan Romance; inilah metrik utama. F1 mikro menghitung seluruh keputusan genre sekaligus. Hamming loss adalah porsi keputusan genre yang salah. Precision@k mengukur berapa dari k genre teratas yang benar, tanpa bergantung pada ambang; metrik ini paling dekat dengan pengalaman pengguna dashboard yang melihat genre terurut.

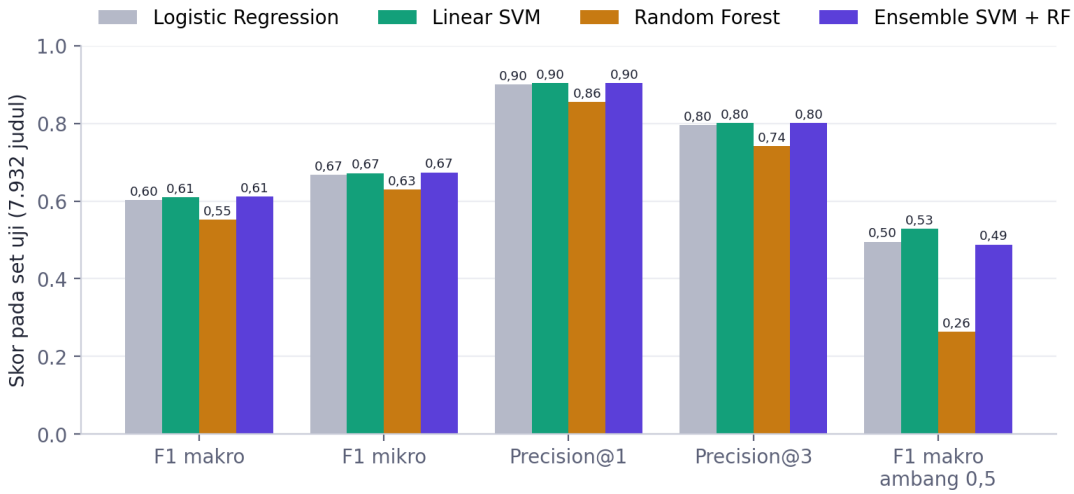
### a. Head A: penandaan genre

Tabel 5.4 dan Gambar 5.3 merangkum hasil Fase 4 pada 7.932 judul uji untuk keempat model. Ensemble SVM + Random Forest mendapat skor OOF tertinggi (0,6059, berbanding 0,6022 untuk Linear SVM, 0,5970 untuk Logistic Regression, dan 0,5492 untuk Random Forest), sehingga terpilih sebagai model Head A. Di set uji, ensemble juga tertinggi pada F1

makro (0,6121), F1 mikro (0,6738), dan Precision@1 (0,9043). Precision@3-nya 0,0008 di bawah Linear SVM. Random Forest sendirian paling lemah, sekitar 0,057 F1 makro di bawah Linear SVM, tetapi kesalahannya berbeda dari SVM sehingga tetap menambah skor saat digabung. Uji lima pembagian data di subbab 5.6 memastikan selisih ini bukan kebetulan satu pembagian.

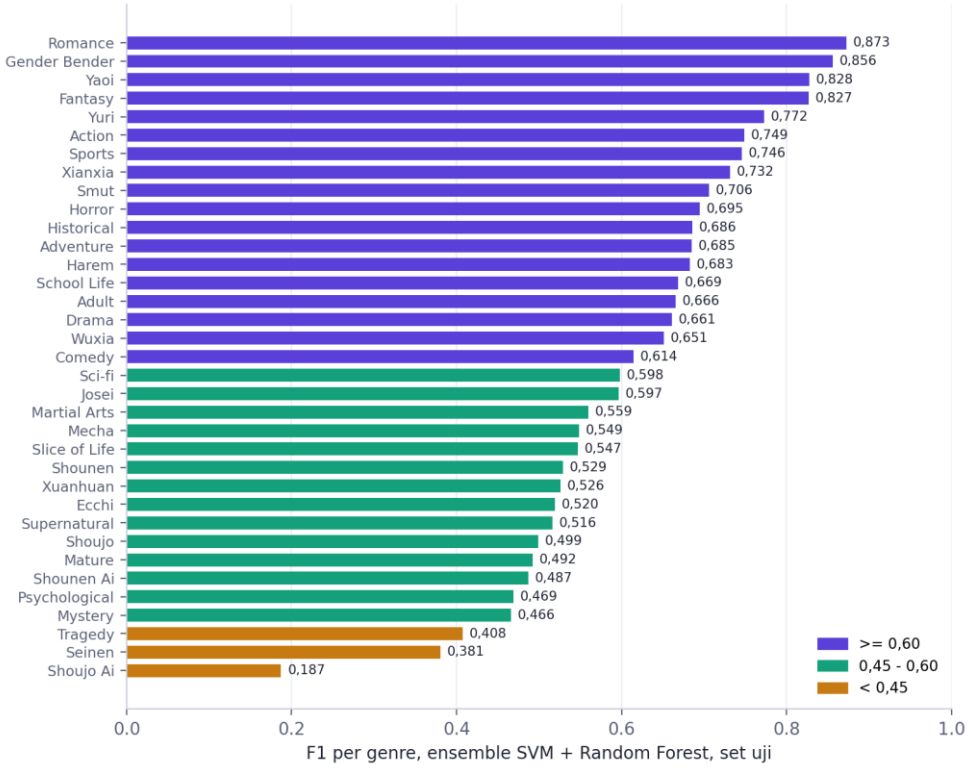
Tabel 5.4 Hasil Head A pada set uji (protokol Fase 4)

| Metrik | Logistic Regression | Linear SVM | Random Forest | Ensemble SVM + RF |
|---|---|---|---|---|
| F1 makro | 0,6027 | 0,6093 | 0,5522 | 0,6121 |
| F1 mikro | 0,6670 | 0,6718 | 0,6296 | 0,6738 |
| F1 per sampel | 0,6554 | 0,6609 | 0,6178 | 0,6624 |
| Hamming loss | 0,1090 | 0,1057 | 0,1239 | 0,1058 |
| Precision@1 | 0,9002 | 0,9032 | 0,8551 | 0,9043 |
| Precision@3 | 0,7953 | 0,8015 | 0,7410 | 0,8007 |
| Precision@5 | 0,7689 | 0,7722 | 0,7130 | 0,7712 |
| F1 makro dengan ambang seragam 0,5 | 0,4954 | 0,5282 | 0,2629 | 0,4881 |
| F1 makro OOF (dasar pemilihan) | 0,5970 | 0,6022 | 0,5492 | 0,6059 |
| Hyperparameter terpilih | C = 0,25, tanpa bobot kelas | C = 0,0354, tanpa bobot kelas | 60 pohon, maks. 2.000 daun, 3.000 fitur chi-kuadrat | bobot SVM 0,7, RF 0,3 (log-odds) |
| Rata-rata ambang per genre | 0,233 | 0,257 | 0,221 | 0,241 |




Gambar 5.3 Perbandingan empat model Head A pada set uji

Gambar 5.4 memperlihatkan F1 per genre untuk ensemble. Rentangnya lebar, dari 0,873 (Romance) sampai 0,187 (Shoujo Ai). Dibanding Linear SVM konfigurasi pertama, F1 naik di 34 dari 35 genre. Kenaikan terbesar ada di genre yang sulit, yaitu Tragedy (+0,045) dan Psychological (+0,045), sedangkan Sports turun 0,004.




Gambar 5.4 F1 per genre pada ensemble SVM + Random Forest (set uji)

### b. Head B: rekomendasi berbasis konten

Tanpa data perilaku, kualitas rekomendasi diukur dengan proksi *genre overlap@10*: dari 10 tetangga terdekat sebuah judul, berapa persen yang berbagi minimal satu genre dengannya. Pada 500 judul acak, nilainya 0,949. Angka ini membuktikan ruang SVD menangkap kemiripan konten, tetapi belum membuktikan pembaca akan menyukai rekomendasinya. Itu baru bisa diukur dengan uji A/B di platform.

### c. Head C: arketipe cerita

Silhouette tertinggi dicapai pada k = 7 (0,0285), dengan nilai untuk k = 5 sampai 10 berturut-turut 0,0261, 0,0266, 0,0285, 0,0276, 0,0274, 0,0267. Nilai di bawah 0,05 berarti klaster saling bertumpuk: ruang konten novel bersifat kontinu, tidak terpisah tegas. Meski begitu, profil klaster mudah dibaca dari istilah dominannya (Tabel 5.5).

Tabel 5.5 Profil tujuh klaster arketipe (Head C)

| Klaster | Judul | Istilah dominan | Genre teratas | Rata-rata WR | Rata-rata bab |
|---|---|---|---|---|---|
| 0. Kehidupan modern dan sekolah | 6.142 | day, modern day, modern, female, girl | Romance, Drama, Comedy | 3,593 | 192 |
| 1. Bangsawan Eropa dan villainess | 3.276 | european ambience, ambience, european, female, ambience female | Romance, Fantasy, Drama | 3,600 | 149 |
| 2. Game, sistem, apokalips | 4.616 | game, world, system, elements, game elements | Fantasy, Action, Adventure | 3,608 | 410 |
| 3. Kultivasi | 4.537 | cultivation, martial, world, sect, immortal | Fantasy, Action, Adventure | 3,570 | 654 |
| 4. Dunia fantasi dan isekai | 5.412 | magic, world, fantasy, another world, another | Fantasy, Action, Adventure | 3,614 | 301 |
| 5. Drama keluarga | 8.683 | one, love, female, family, day | Romance, Drama, Fantasy | 3,592 | 175 |

| Klaster | Judul | Istilah dominan | Genre teratas | Rata-rata WR | Rata-rata bab |
|---|---|---|---|---|---|
| 6. Romansa dan love interest | 6.994 | love, love interests, interests, first, love first | Romance, Drama, Fantasy | 3,812 | 257 |

### d. Head D: estimasi rating

Pada 4.652 judul uji, Head D mencapai MAE 0,3250 dan RMSE 0,4157, dibandingkan baseline yang selalu menebak rata-rata dengan MAE 0,4151. Galat absolut turun 21,7%. R2 sebesar 0,354 berarti fitur konten menjelaskan sekitar 35% variasi rating (Tabel 5.6). Konfigurasi pertama (SVD dan fitur numerik saja, tanpa bobot sampel) yang dilatih ulang pada pembagian yang sama mencapai MAE 0,3395, jadi skor ridge, tag, bobot suara, dan parameter hasil TPE bersama-sama menurunkan MAE 0,0145. Untuk evaluasi ini, TF-IDF dan SVD di-*fit* ulang tanpa baris uji, karena representasi bersama yang dipakai dashboard di-*fit* pada seluruh katalog.

Tabel 5.6 Hasil Head D pada set uji

| Model dan fitur | MAE | RMSE | R2 |
|---|---|---|---|
| Baseline: rata-rata rating latih | 0,4151 | 0,5174 | 0,0000 |
| HistGradientBoosting, SVD 200 + 20 numerik (Fase 4, konfigurasi pertama) | 0,3395 | 0,4307 | 0,3071 |
| Ridge bertumpuk + HistGradientBoosting, SVD 200 + 20 numerik + 678 tag, bobot suara (Fase 4) | 0,3250 | 0,4157 | 0,3544 |
| HistGradientBoosting, 205 fitur tabular (Fase 5) | 0,3303 | 0,4222 | 0,3342 |
| Spark MLlib GBTRegressor, 205 fitur tabular (Fase 5) | 0,3397 | 0,4347 | 0,2939 |

### e. Loss pelatihan dan pengujian

Metrik F1 dan Precision@k mengukur keputusan akhir, sedangkan loss mengukur seberapa yakin dan seberapa tepat probabilitas model. Untuk Head A dipakai *binary cross-entropy* (log loss) yang dirata-ratakan atas 35 genre, dihitung pada set latih, validasi, dan uji dengan protokol Fase 5 sehingga keempat model pada Tabel 5.3 dapat dibandingkan langsung (Tabel 5.7). Selisih antara loss latih dan loss uji menunjukkan seberapa jauh model menghafal data latih.

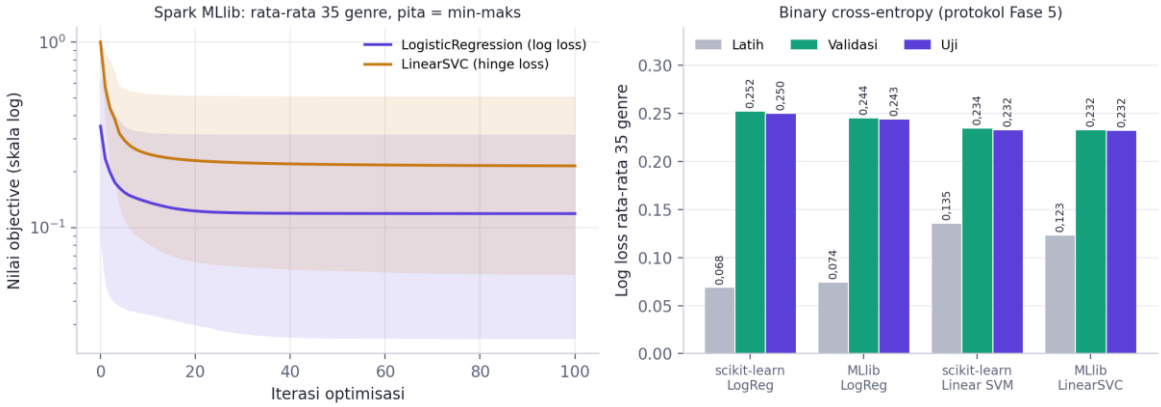
Tabel 5.7 Loss Head A pada set latih, validasi, dan uji

| Model | Log loss latih | Log loss validasi | Log loss uji | Selisih uji - latih | Hamming loss uji |
|---|---|---|---|---|---|
| scikit-learn Logistic Regression | 0,0683 | 0,2516 | 0,2496 | 0,1813 | 0,1175 |
| Spark MLlib LogisticRegression | 0,0739 | 0,2445 | 0,2435 | 0,1696 | 0,1178 |
| scikit-learn Linear SVM | 0,1352 | 0,2340 | 0,2324 | 0,0972 | 0,1142 |
| Spark MLlib LinearSVC | 0,1226 | 0,2323 | 0,2319 | 0,1093 | 0,1145 |

Pada keempat model, log loss latih jauh lebih rendah daripada log loss validasi dan uji. Logistic Regression scikit-learn misalnya turun ke 0,0683 pada data latih, sedangkan pada data uji 0,2496. Celah ini wajar untuk model linear dengan sekitar 90 ribu fitur dan menunjukkan model sebagian menghafal kata-kata khas judul latih. Log loss validasi dan uji hampir sama untuk semua model, sehingga ambang yang disetel pada data validasi dapat dipercaya berlaku juga pada data uji. Model dengan log loss uji terendah adalah LinearSVC MLlib (0,2319) dan Linear SVM scikit-learn (0,2324), sejalan dengan F1 makro keduanya yang tertinggi. LogisticRegression MLlib punya log loss uji sedikit lebih rendah (0,2435) daripada padanannya

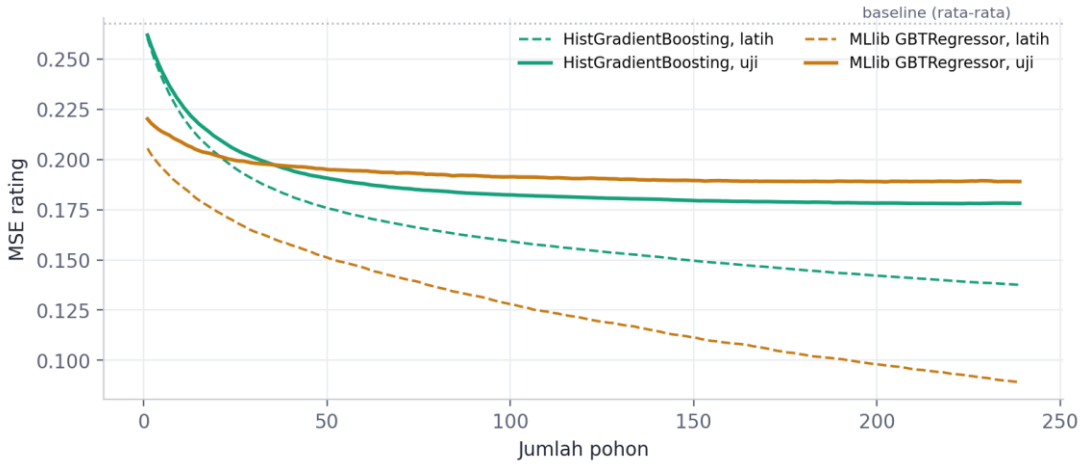
di scikit-learn (0,2496) walaupun F1 makronya sedikit lebih rendah; probabilitasnya sedikit lebih terkalibrasi, tetapi urutan genrenya sedikit kurang tajam.

Kurva *objective* di Gambar 5.5 (kiri) adalah nilai fungsi yang diminimalkan MLlib di setiap iterasi: log loss ditambah regularisasi untuk LogisticRegression, dan *hinge loss* berbobot ditambah regularisasi untuk LinearSVC. Untuk LogisticRegression, rata-rata objective turun dari 0,3513 ke 0,1187, dan 98% penurunannya terjadi pada 20 iterasi pertama. Ke-35 genre menyentuh batas 100 iterasi tanpa memenuhi toleransi konvergensi bawaan, tetapi 10 iterasi terakhir hanya menurunkan objective kurang dari 0,01% (median), jadi model sudah praktis konvergen. LinearSVC turun dari 1,0000 ke 0,2148 dan masih bergerak sedikit di akhir (0,26% pada 10 iterasi terakhir). Menaikkan batas iterasi hanya akan menambah waktu latih yang sudah paling panjang di antara keempat model.




Gambar 5.5 Kurva objective MLlib per iterasi (kiri) dan log loss latih, validasi, dan uji Head A (kanan)

Untuk Head D, loss yang dipantau pada protokol Fase 5 (205 fitur tabular yang identik untuk kedua kerangka) adalah *mean squared error* (MSE) rating pada setiap penambahan pohon (Gambar 5.6). Baseline yang selalu menebak rata-rata memberi MSE 0,2677. HistGradientBoosting berhenti di 239 pohon dengan MSE latih 0,1376 dan MSE uji 0,1782; MSE uji terendahnya 0,1780, jadi titik henti *early stopping* hampir tepat di titik terbaik. GBTRegressor MLlib dengan jumlah pohon yang sama mencapai MSE latih 0,0891 tetapi MSE uji 0,1890. Celah latih-uji yang lebih lebar ini menandakan *overfitting*: HistGradientBoosting pada konfigurasi pembanding ini memakai regularisasi L2 pada daun (`l2_regularization` = 1,0) dan *binning* histogram, sedangkan GBT MLlib tidak punya regularisasi setara.




Gambar 5.6 Kurva MSE latih dan uji Head D per jumlah pohon

## 5.6 Analisis

**a. Penyetelan ambang memberi peningkatan terbesar**

Dengan ambang seragam 0,5, F1 makro ensemble hanya 0,4881. Dengan ambang per genre, nilainya naik menjadi 0,6121, peningkatan 0,1240. Peningkatan pada Linear SVM 0,0811 (0,5282 menjadi 0,6093) dan pada Logistic Regression 0,1073 (0,4954 menjadi 0,6027). Random Forest paling bergantung pada ambang: tanpa penyetelan F1 makronya hanya 0,2629, karena rata-rata suara pohon untuk genre langka jarang mencapai 0,5. Rata-rata ambang terpilih hanya 0,241 untuk ensemble, jauh di bawah 0,5, karena sebagian besar genre adalah kelas minoritas. Di antara tiga model terbaik, pilihan algoritma hanya menggeser F1 makro sekitar 0,009, sedangkan ambang menggesernya lebih dari 0,08.

**b. Penyetelan ulang menyumbang lebih banyak daripada algoritma baru**

Dibanding konfigurasi pertama (Linear SVM dengan F1 makro uji 0,5913), model Head A sekarang naik 0,021 menjadi 0,6121. Sebagian besar kenaikan berasal dari penyetelan ulang C dan bobot blok fitur, bukan dari ukuran kosakata atau algoritma baru: Linear SVM dengan kosakata 40.000 yang lama sudah mencapai 0,6101 setelah C dan bobotnya disetel. Agar tidak bergantung pada satu pembagian, konfigurasi lama dan baru dilatih ulang pada lima pembagian data, lalu selisihnya diuji dengan uji t terkoreksi Nadeau dan Bengio (2003), yang memperhitungkan tumpang tindih data latih antar pembagian. Linear SVM baru unggul 0,0197 F1 makro dan ensemble unggul 0,0231 atas Linear SVM lama, keduanya di kelima pembagian ($p < 0,0001$). Ensemble juga unggul 0,0034 atas Linear SVM baru di kelima pembagian ($p = 0,016$).

Pemulihan kata bersensor hampir tidak menggeser skor uji (0,5911 berbanding 0,5913), karena token yang rusak muncul dengan cara yang sama di data latih dan data uji. Manfaatnya terlihat ketika masukan berupa kata utuh, seperti teks yang diketik pengguna di dashboard. Pada 2.352 judul uji yang terkena sensor, dengan sinopsis versi utuh sebagai masukan, model yang dilatih dari teks yang dipulihkan mencapai F1 makro 0,6014, sedangkan model yang dilatih dari teks bersensor 0,5946. Metode lain dari literatur juga diuji pada pembagian yang sama dan tidak dipakai karena tidak lebih baik: Okapi BM25, n-gram 1 sampai 3, NB-SVM, Naive Bayes, kalibrasi isotonic, pencarian ambang yang lebih halus, classifier chains (Read et al., 2011), dan stacking antar genre.

**c. Model cukup andal sebagai saran, belum sebagai keputusan otomatis**

Precision@1 sebesar 0,904 berarti genre teratas yang disarankan model benar pada sekitar 9 dari 10 judul, dan Precision@3 sebesar 0,801. Namun F1 makro 0,612 menunjukkan model masih sering melewatkan atau menambah genre. Alur kerja yang disarankan untuk tim kurasi adalah menampilkan tiga genre teratas sebagai saran yang dikonfirmasi manusia. Genre yang F1-nya rendah, seperti Shoujo Ai, Seinen, dan Tragedy, membutuhkan perhatian khusus; Bab 6 menjelaskan mengapa genre tersebut sulit.

**d. Estimasi rating hanya berguna sebagai prior**

Head D menurunkan MAE 21,7% dari baseline, tetapi R2 0,35 menunjukkan sebagian besar variasi rating tidak bisa dijelaskan dari isi katalog. Faktor seperti kualitas terjemahan dan popularitas penulis tidak ada di dataset. Estimasi ini cocok sebagai nilai awal untuk judul yang belum punya cukup suara, bukan pengganti rating pembaca.

**e. Kapan Spark MLlib layak dipakai**

Pada ukuran data saat ini, scikit-learn belasan sampai puluhan kali lebih cepat dengan kualitas yang setara, sehingga tetap dipakai sebagai jalur produksi. Keputusan ini berlaku selama data latih muat di memori driver. Matriks fitur Head A berukuran 39.660 x 165.010,

tetapi hanya sekitar 290 sel per baris yang terisi, sehingga ukurannya sekitar 130 MB. Ketika katalog diperluas ke sekitar 76 ribu judul yang sedang di-scrape, ukurannya kira-kira dua kali lipat dan masih aman.

Jalur MLlib tetap bernilai karena dua hal. Pertama, ia membuktikan seluruh pipeline dapat berjalan tanpa `toPandas()`: fitur dan model dibangun langsung di atas data di lake, sehingga batas ukurannya adalah kapasitas klaster, bukan memori satu mesin. Kedua, pengembangannya menghasilkan pelajaran yang tidak terlihat bila hanya membaca dokumentasi: normalisasi L2 harus ditambahkan secara eksplisit, GBT butuh checkpoint, dan early stopping MLlib perlu disetel manual. Jalur ini disiapkan untuk dipakai bila data latih melampaui memori driver atau bila klaster ditambah worker.

# BAB 6
# GRAPH ANALYTICS

Assignment I sudah membangun *property graph* katalog (67.799 simpul, 858.901 sisi) dan menganalisis dua proyeksinya: ko-okurensi genre dan kolaborasi penulis (Tabel 4.6). Kedua graf itu dibentuk dari metadata. Bab ini menambahkan graf ketiga yang dibentuk dari keluaran pipeline *machine learning* di Bab 5, sehingga *graph analytics* dapat dipakai untuk menjelaskan perilaku model.

## 6.1 Pengembangan *Graph*

### a. Graf kemiripan novel (kNN)

Setiap judul menjadi simpul, dan setiap simpul dihubungkan ke 10 tetangga terdekatnya menurut cosine similarity di ruang SVD 200 dimensi yang juga dipakai Head B. Sisi diberi bobot nilai cosine-nya. Pasangan (a, b) dan (b, a) digabung karena graf tidak berarah, sehingga terbentuk 314.786 sisi dengan bobot rata-rata 0,584. Matriks kemiripan penuh 39.660 x 39.660 bernilai float32 membutuhkan sekitar 6,3 GB, jadi tetangga dihitung per potongan 2.048 baris dan hanya 10 nilai teratas per baris yang disimpan.

### b. Algoritma graf

Empat analisis dijalankan dengan NetworkX. *Connected components* memeriksa apakah katalog terpecah menjadi pulau-pulau. PageRank berbobot mencari judul yang menjadi pusat ruang konten. Deteksi komunitas Louvain (Blondel et al., 2008) mencari kelompok judul yang lebih rapat ke dalam daripada ke luar. Terakhir, homofili per genre (McPherson et al., 2001) mengukur berapa persen tetangga judul bergenre G yang juga bergenre G. Hasil komunitas dibandingkan dengan klaster K-Means Head C dan genre utama memakai *normalized mutual information* (NMI) dan *adjusted Rand index* (ARI) (Vinh et al., 2010). Implementasinya ada di `src/phases/similarity_graph.py` (Pseudocode 6.1). Ukuran graf dan hasil keempat analisis itu dirangkum pada Tabel 6.1.

**Pseudocode 6.1 Pembangunan dan analisis graf kemiripan**

```
// source: src/phases/similarity_graph.py
E <- L2-normalized SVD embeddings (39,660 x 200)          // Head B space
FOR each block of 2,048 rows:
    S <- block @ transpose(E); S[i, i] <- -infinity
    neighbours <- 10 largest columns of S per row
edges <- undirected union of (i, j, cosine), keep the max weight
G <- Graph(39,660 nodes, edges)                           // 314,786 edges
components  <- connected_components(G)
pr          <- pagerank(G, weight = cosine)
communities <- louvain(G, weight = cosine, seed = 42)
agreement   <- NMI, ARI(communities, kmeans_cluster | primary_genre)
FOR genre g: share[g] <- neighbours of g-titles that also carry g / all neighbours
rho <- spearman(share, F1 per genre of the selected Head A model)
```

Tabel 6.1 Ringkasan graf kemiripan novel

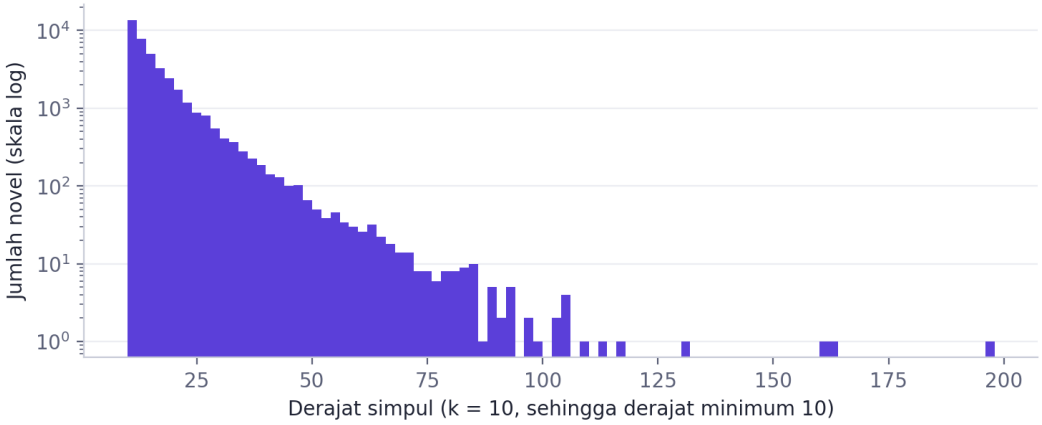
| Ukuran | Nilai |
|---|---:|
| Simpul / sisi | 39.660 / 314.786 |
| Derajat minimum / median / rata-rata | 10 / 13 / 15,87 |
| Derajat persentil 99 / maksimum | 50 / 198 |
| Komponen terhubung | 1 (seluruh 39.660 simpul) |
| Komunitas Louvain / dengan >= 100 judul | 27 / 26 |
| Modularity | 0,7192 |
| NMI komunitas vs klaster K-Means | 0,3414 |
| ARI komunitas vs klaster K-Means | 0,1955 |

| Ukuran | Nilai |
|---|---|
| NMI komunitas vs genre utama | 0,1145 |
| NMI klaster K-Means vs genre utama | 0,1095 |
| Spearman PageRank vs jumlah suara | 0,0308 |

## 6.2 Analisis dan *Insight* yang Diperoleh

### a. Pusat graf bukan judul populer

Graf ini terhubung penuh dalam satu komponen. Derajat minimumnya 10 karena setiap simpul memilih 10 tetangga, tetapi distribusinya berekor panjang: median 13, sedangkan satu judul terhubung dengan 198 judul lain (Gambar 6.1). Gejala ini dikenal sebagai *hubness* pada data berdimensi tinggi (Radovanovic et al., 2010).

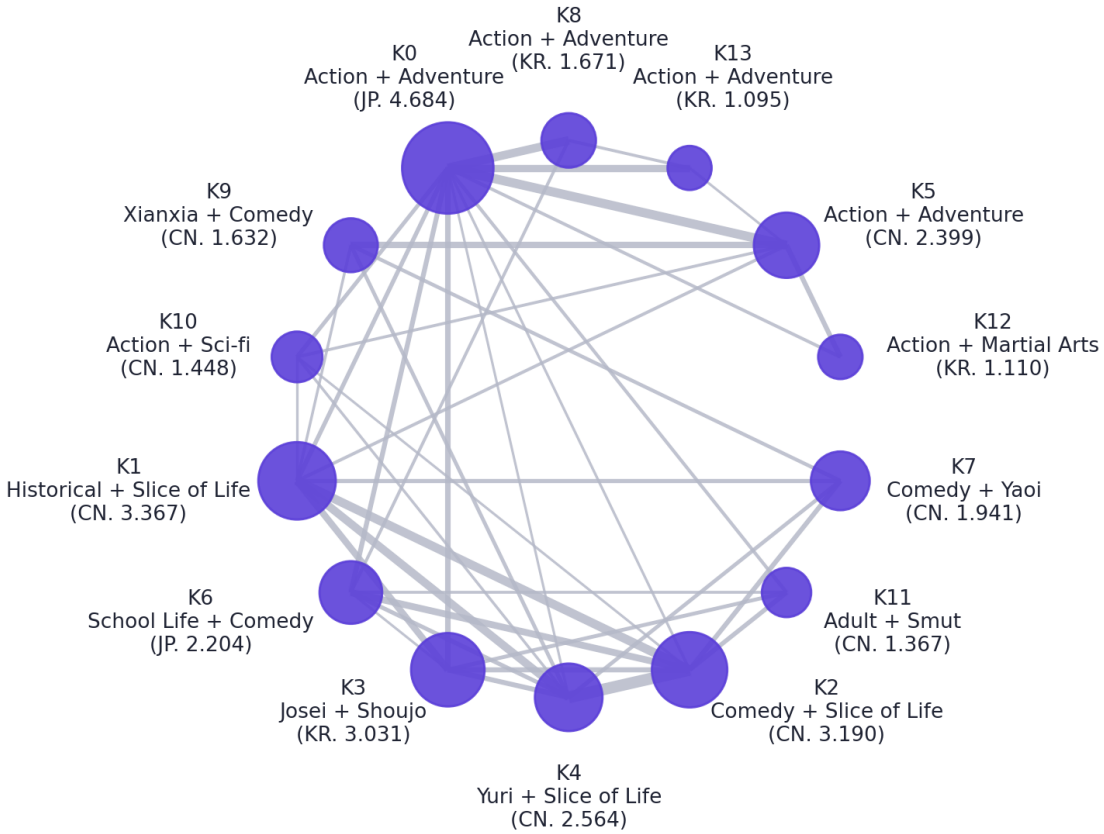



Gambar 6.1 Distribusi derajat simpul pada graf kemiripan (skala log)

Judul dengan PageRank tertinggi bukan judul populer. Tiga teratas adalah My Partner Is Not Human [Quick Transmigration] (0 suara); After the Bankrupt Heiress Married into a Rich Family (6 suara); The Cold, Aloof Senior Sister Deserves To Be Paired With A Peerless Beauty (4 suara). Korelasi Spearman antara PageRank dan jumlah suara hanya 0,031. Pusat graf adalah judul dengan sinopsis dan tag yang sangat "rata-rata", misalnya kultivasi atau *quick transmigration* dengan kosakata templat. *Insight*-nya untuk Head B: tanpa koreksi, judul pusat semacam ini akan muncul di daftar rekomendasi banyak judul lain walaupun kualitasnya belum terbukti. Rekomendasi sebaiknya diurutkan ulang dengan weighted rating, atau tetangga dihitung secara mutual agar judul pusat tidak mendominasi.

### b. Komunitas lebih tajam daripada klaster K-Means

Louvain menemukan 27 komunitas dengan modularity 0,719, jauh di atas modularity graf ko-okurensi genre di Assignment I (0,0858). Artinya judul membentuk kelompok konten yang jelas, walaupun label genrenya saling berbaur. Gambar 6.2 memperlihatkan 14 komunitas terbesar; label tiap komunitas adalah dua genre terbanyak di luar Romance, Fantasy, dan Drama yang ada di hampir semua komunitas.




Gambar 6.2 Empat belas komunitas terbesar graf kemiripan; tebal garis menunjukkan jumlah sisi antarkomunitas (>= 400)

Tabel 6.2 Profil delapan komunitas terbesar

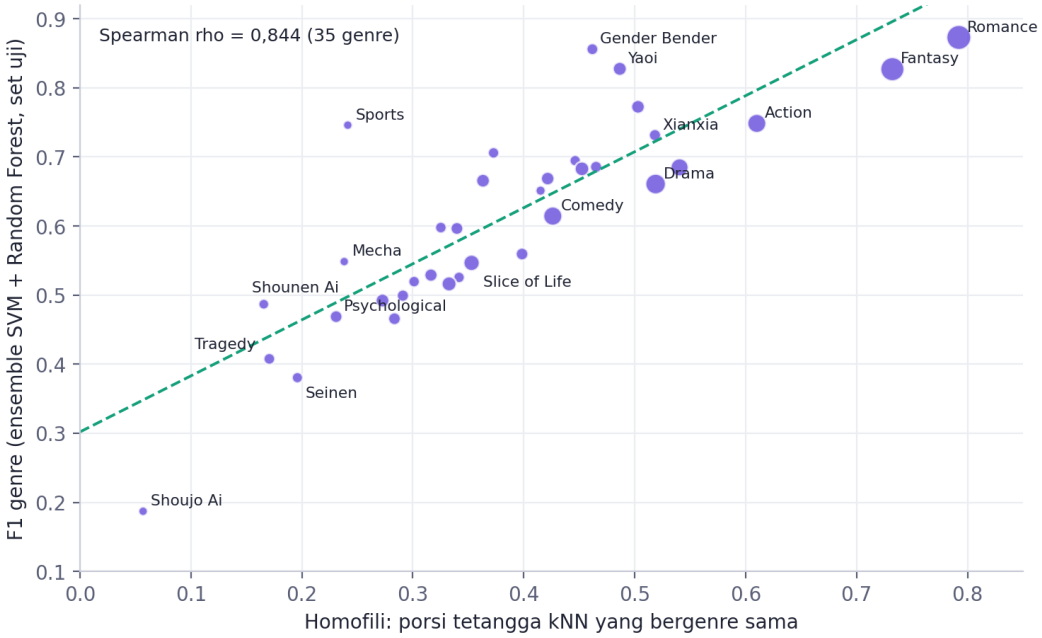
| Komunitas | Judul | Genre terbanyak | Asal dominan | Rata-rata WR | Contoh judul populer |
|---|---|---|---|---|---|
| K0 | 4.684 | Fantasy, Action, Adventure, Comedy | JP (42,3%) | 3,624 | Release that Witch |
| K1 | 3.367 | Romance, Drama, Historical, Fantasy | CN (88,6%) | 3,692 | The Founder of Diabolism |
| K2 | 3.190 | Romance, Drama, Comedy, Slice of Life | CN (79,7%) | 3,777 | My Husband is Suffering from a Terminal Illness |
| K3 | 3.031 | Romance, Fantasy, Drama, Josei | KR (84,1%) | 3,626 | The Reader and Protagonist Definitely Have to Be in True Love |
| K4 | 2.564 | Romance, Drama, Yuri, Fantasy | CN (79,1%) | 3,580 | A Game To Make Him Fall |
| K5 | 2.399 | Action, Fantasy, Adventure, Martial Arts | CN (88,9%) | 3,542 | Against the Gods |
| K6 | 2.204 | Romance, School Life, Comedy, Drama | JP (67,3%) | 3,606 | About the Reckless Girl Who Kept Challenging a Reborn Man Like Me |
| K7 | 1.941 | Romance, Fantasy, Drama, Comedy | CN (96,8%) | 3,657 | Quickly Wear the Face of the Devil |

Komunitas memisahkan judul menurut gabungan asal negara dan latar cerita, bukan menurut genre utama (Tabel 6.2). K0 dan K5 sama-sama didominasi Action dan Adventure, tetapi asal terbanyak di K0 adalah Jepang (42,3%) dengan tema dunia lain, sedangkan K5 hampir seluruhnya xianxia dan kultivasi Tiongkok (88,9% CN). Romansa terbelah menjadi romansa

sejarah Tiongkok (K1, 88,6% CN), romansa modern Tiongkok (K2) yang punya rata-rata WR tertinggi di antara delapan komunitas terbesar (3,777), dan romansa fantasi Korea (K3, 84,1% KR). NMI komunitas terhadap genre utama hanya 0,115, setara dengan K-Means (0,110). Kesepakatan komunitas dengan K-Means sedang (NMI 0,341): keduanya menangkap struktur yang serupa, tetapi Louvain memecahnya lebih halus. Bagi tim lisensi, segmentasi yang lebih berguna adalah komunitas seperti ini daripada genre utama, karena judul di satu komunitas bersaing memperebutkan pembaca yang sama.

**c. Homofili menjelaskan genre yang sulit diprediksi**

Gambar 6.3 menghubungkan graf dengan hasil Bab 5. Sumbu mendatar adalah homofili, yaitu porsi tetangga kNN yang bergenre sama; sumbu tegak adalah F1 genre itu pada ensemble SVM + Random Forest, model Head A yang terpilih. Korelasi Spearman-nya 0,844 ($p < 0,001$, 35 genre). Korelasi ini bukan sekadar efek ukuran genre: setelah prevalensi genre dikendalikan, korelasi parsialnya tetap 0,857. Prevalensi sendiri hanya berkorelasi 0,348 dengan F1.




Gambar 6.3 Homofili genre di graf kemiripan terhadap F1 per genre; ukuran titik sebanding dengan jumlah judul

Genre yang judulnya saling berdekatan di ruang konten, seperti Romance (79,2% tetangga sesama Romance) dan Action (61,0%), mudah diprediksi. Genre yang judulnya tersebar justru sulit: hanya 5,7% tetangga judul Shoujo Ai yang juga Shoujo Ai, 16,6% untuk Shounen Ai, dan 19,6% untuk Seinen. Seinen, Shoujo, Shounen, dan Josei adalah label demografi pembaca sasaran, bukan isi cerita, sehingga sinopsisnya tidak membawa ciri yang konsisten. Tragedy dan Psychological bersifat suasana, yang juga jarang tertulis eksplisit di sinopsis.

Dua genre berada jauh di atas garis: Sports dan Gender Bender. Homofilinya sedang, tetapi F1-nya tinggi karena keduanya hampir selalu disertai tag yang sangat spesifik. Model menangkap tag itu walaupun sinopsisnya mirip judul lain.

**d. Implikasi**

Analisis graf memberi tiga keputusan yang tidak terlihat dari metrik model saja:

- **Kurasi manual dipusatkan pada genre berhomofili rendah.** Prediksi Shoujo Ai, Shounen Ai, Tragedy, Seinen, dan Psychological sebaiknya hanya menjadi saran yang wajib

dikonfirmasi kurator. Untuk genre berhomofili tinggi, prediksi model cukup diaudit secara sampel.

- **Rekomendasi perlu koreksi hubness.** Judul pusat graf punya sedikit suara, sehingga pengurutan ulang dengan weighted rating atau kNN mutual mencegah judul templat membanjiri rekomendasi.
- **Segmentasi pasar memakai komunitas, bukan genre utama.** Komunitas menggabungkan asal negara dan latar cerita, dua hal yang menentukan pembaca sasaran dan biaya lisensi, dan modularity-nya (0,719) jauh lebih tegas daripada graf genre (0,0858).

# REFERENSI


- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. Dalam Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (hlm. 2623-2631). ACM.
- Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. Dalam Advances in Neural Information Processing Systems 24 (hlm. 2546-2554).
- Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. Journal of Statistical Mechanics: Theory and Experiment, 2008(10), P10008.
- Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.
- Clauset, A., Newman, M. E. J., & Moore, C. (2004). Finding community structure in very large networks. *Physical Review E*, 70(6), 066111.
- Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1), 107-113.
- Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by latent semantic analysis. Journal of the American Society for Information Science, 41(6), 391-407.
- Fan, R.-E., & Lin, C.-J. (2007). A study on threshold selection for multi-label classification. Technical report, National Taiwan University.
- Fan, R.-E., Chang, K.-W., Hsieh, C.-J., Wang, X.-R., & Lin, C.-J. (2008). LIBLINEAR: A library for large linear classification. Journal of Machine Learning Research, 9, 1871-1874.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189-1232.
- Genest, C., Weerahandi, S., & Zidek, J. V. (1984). Aggregating opinions through logarithmic pooling. Theory and Decision, 17(1), 61-70.
- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley.
- Madjarov, G., Kocev, D., Gjorgjevikj, D., & Džeroski, S. (2012). An extensive experimental comparison of methods for multi-label learning. Pattern Recognition, 45(9), 3084-3104.
- McPherson, M., Smith-Lovin, L., & Cook, J. M. (2001). Birds of a feather: Homophily in social networks. Annual Review of Sociology, 27, 415-444.
- Meng, X., Bradley, J., Yavuz, B., Sparks, E., et al. (2016). MLlib: Machine learning in Apache Spark. Journal of Machine Learning Research, 17(34), 1-7.
- Nadeau, C., & Bengio, Y. (2003). Inference for the generalization error. Machine Learning, 52(3), 239-281.
- Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab.
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

- Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Dalam Advances in Large Margin Classifiers (hlm. 61-74). MIT Press.
- Radovanovic, M., Nanopoulos, A., & Ivanovic, M. (2010). Hubs in space: Popular nearest neighbors in high-dimensional data. Journal of Machine Learning Research, 11, 2487-2531.
- Read, J., Pfahringer, B., Holmes, G., & Frank, E. (2011). Classifier chains for multi-label classification. Machine Learning, 85(3), 333-359.
- Republik Indonesia. (2014). Undang-Undang Nomor 28 Tahun 2014 tentang Hak Cipta.
- Republik Indonesia. (2022). Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, 20, 53-65.
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513-523.
- Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.
- Vinh, N. X., Epps, J., & Bailey, J. (2010). Information theoretic measures for clusterings comparison. Journal of Machine Learning Research, 11, 2837-2854.
- Wolpert, D. H. (1992). Stacked generalization. Neural Networks, 5(2), 241-259.
- Zaharia, M., Xin, R. S., Wendell, P., et al. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56-65.
- Zhang, M.-L., & Zhou, Z.-H. (2014). A review on multi-label learning algorithms. IEEE Transactions on Knowledge and Data Engineering, 26(8), 1819-1837.
- NovelUpdates. Direktori web novel terjemahan. https://www.novelupdates.com


# Link Penting:

## Link Drive:

**https://drive.google.com/drive/folders/1BzosnHOG89Nuuv8qC_OKu9b62uEUXT_T?usp=sharing**

## Link Video:

**https://drive.google.com/file/d/164RRwafO2FbzZ0Aa-AsWX7xVmm-XlP4q/view?usp=drive_link**

## Link PPT:

https://docs.google.com/presentation/d/1x8pjQd03h27ClxkmZfYGhRdN6U3ZB35U/edit?usp=drive_link&ouid=107313197034562171465&rtpof=true&sd=true

**Link Deklarasi AI:**

https://drive.google.com/file/d/1fx9rQLxQrt5F3UaOw-TYeDRwGaOlZeIu/view?usp=sharing